

5.5.3 verilog 语言的描述风格

三种描述风格：

行为描述： `always`语句，使用功能描述

数据流描述： `assign`语句，使用布尔代数式描述

结构化描述： 元件例化

5.5.4 基本逻辑电路的VHDL设计

一、组合电路

原则1：在always中用到的所有输入信号都出现在敏感信号列表中；

原则2：电路的真值表必须在代码中完整的反映出来（使用default），否则会生成锁存器；

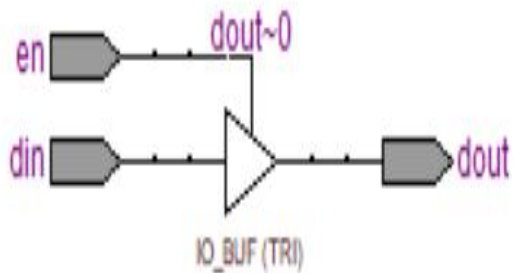
原则3：使用if · · else · · ，必须完全互斥；

原则4：不能在一个表达式中对某个信号即读又写（不能反馈）；

原则5：不能使用 <= 非阻塞赋值。

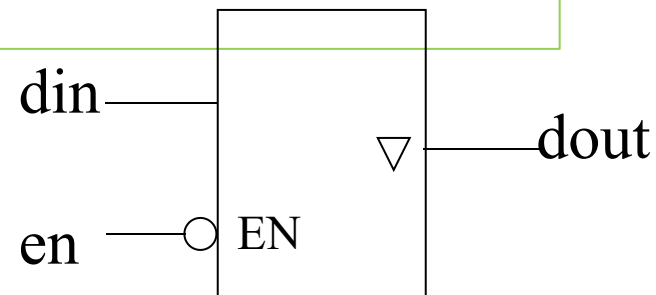
1. 三态门及总线缓冲器

```
module tri (  
    input    din ,  
    input    en ,  
    output   dout  
);  
  
    assign dout = en ? din : 1'bz;  
  
endmodule
```



8位数据总线?

```
module tri (  
    input    din ,  
    input    en ,  
    output reg   dout  
);  
  
    always @ ( * ) begin  
        if ( !en)  
            dout = din ;  
        else  
            dout = 1'bz ;  
    end  
  
endmodule
```



2. 3: 8译码器

```
module decode3_8(  
    input      G1 ,  
    input      G2 ,  
    input      G3 ,  
    input [2:0] A ,  
    output reg [7:0] Y  
);
```

```
reg s;
```

```
always @ ( * ) begin
```

```
    s = G2 | G3 ;
```

```
    if (G1 == 0 || s == 1)
```

```
        Y = 8'b1111_1111;
```

```
    else begin
```

```
        case ( A )
```

```
            3'b000:    Y = 8'b1111_1110;
```

```
            3'b001:    Y = 8'b1111_1101;
```

```
            3'b010:    Y = 8'b1111_1011;
```

```
            3'b011:    Y = 8'b1111_0111;
```

```
            3'b100:    Y = 8'b1110_1111;
```

```
            3'b101:    Y = 8'b1101_1111;
```

```
            3'b110:    Y = 8'b1011_1111;
```

```
            3'b111:    Y = 8'b0111_1111;
```

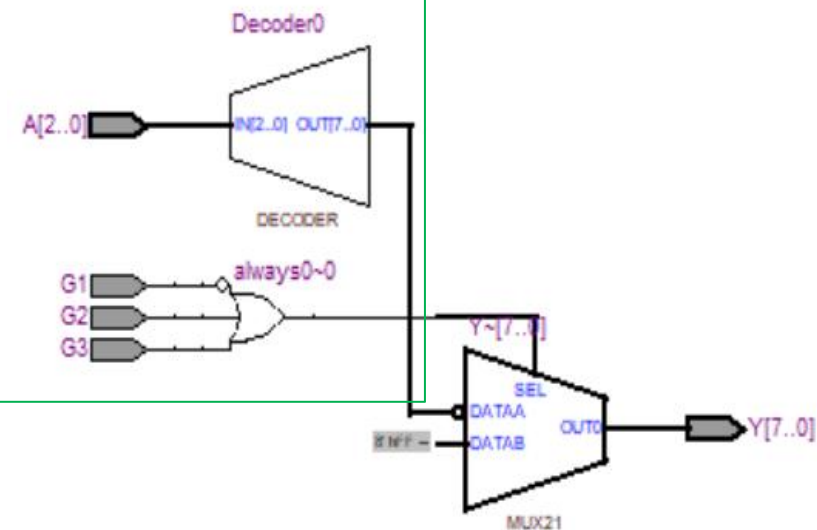
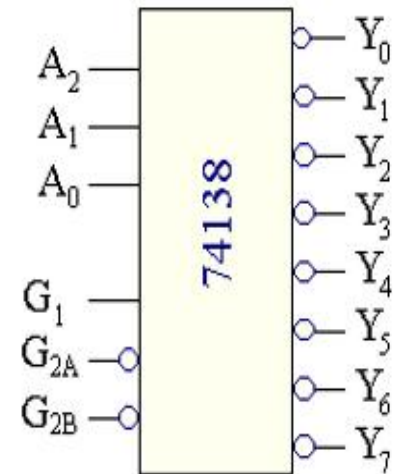
```
            default:    Y = 8'b1111_1111;
```

```
        endcase
```

```
    end
```

```
end
```

```
endmodule
```



使能端 if

3. 8-3优先编码器

```

module encode_83(
    input [7:0]      I ,
    input           EI ,
    output reg [2:0] A ,
    output reg     GS ,
    output reg     EO
);

always @ ( * ) begin
    if (EI) begin
        A = 3'b111 ;
        GS = 1'b1 ;
        EO = 1'b1 ;
    end

```

```

    else if (I[7] == 0) begin
        A = 3'b000;
        GS = 1'b0;
        EO = 1'b1 ;
    end else if (I[6] == 0) begin

```

.....

```

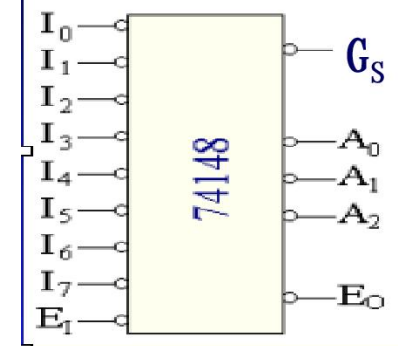
    end else if (I[0] == 0) begin
        A = 3'b111;
        GS = 1'b 0;
        EO = 1'b1;
    end else begin
        A = 3'b111;
        GS = 1'b1;
        EO = 1'b0;
    end

```

```

end
endmodule

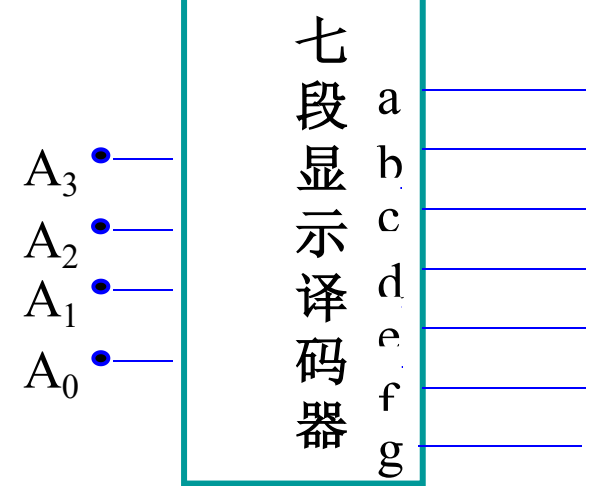
```



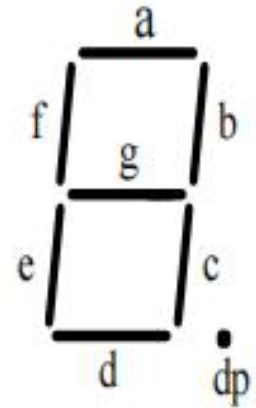
$\overline{E_1}$	输 入								输 出			
	$\bar{I}_7$	$\bar{I}_6$	$\bar{I}_5$	$\bar{I}_4$	$\bar{I}_3$	$\bar{I}_2$	$\bar{I}_1$	$\bar{I}_0$	$\bar{Y}_2$	$\bar{Y}_1$	$\bar{Y}_0$	$\overline{G_s E_o}$
1	×	×	×	×	×	×	×	×	1	1	1	1 1
0	1	1	1	1	1	1	1	1	1	1	1	1 0
0	0	×	×	×	×	×	×	×	0	0	0	0 1
0	1	0	×	×	×	×	×	×	0	0	1	0 1
0	1	1	0	×	×	×	×	×	0	1	0	0 1
0	1	1	1	0	×	×	×	×	0	1	1	0 1
0	1	1	1	1	0	×	×	×	1	0	0	0 1
0	1	1	1	1	1	0	×	×	1	0	1	0 1
0	1	1	1	1	1	1	0	×	1	1	0	0 1
0	1	1	1	1	1	1	1	0	1	1	1	0 1

4. 七段显示译码器

```
module seg_7 (  
    input [3:0]      A ,  
    output reg [7:0] seg  
);  
  
always @ ( * ) begin  
    case (A)          //a段是最低位  
        4'd0: seg = 8'b0011_1111 ;  
        4'd1: seg = 8'b0000_0110 ;  
        4'd2: seg = 8'b0101_1011 ;  
        4'd3: seg = 8'b"0100_1111 ;  
        4'd4: seg = 8'b0110_0110 ;  
        4'd5: seg = 8'b0110_1101 ;  
        4'd6: seg = 8'b0111_1110 ;  
    endcase  
end  
endmodule
```



510Ω × 7



a是最低位

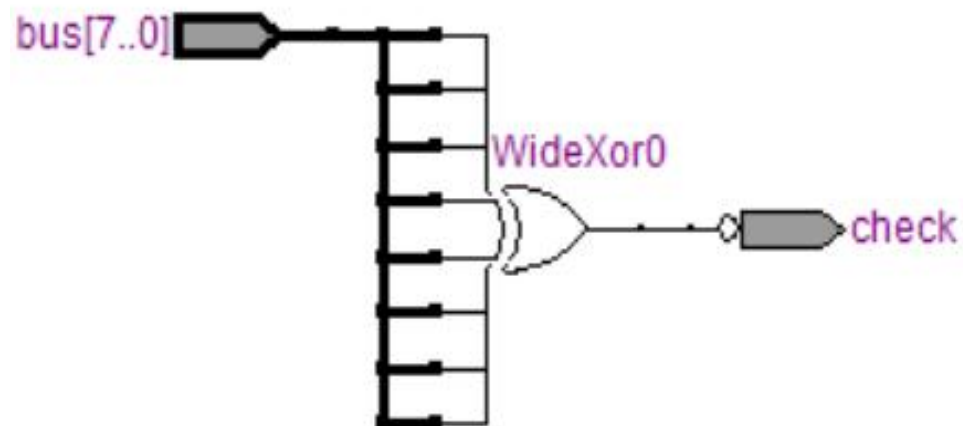
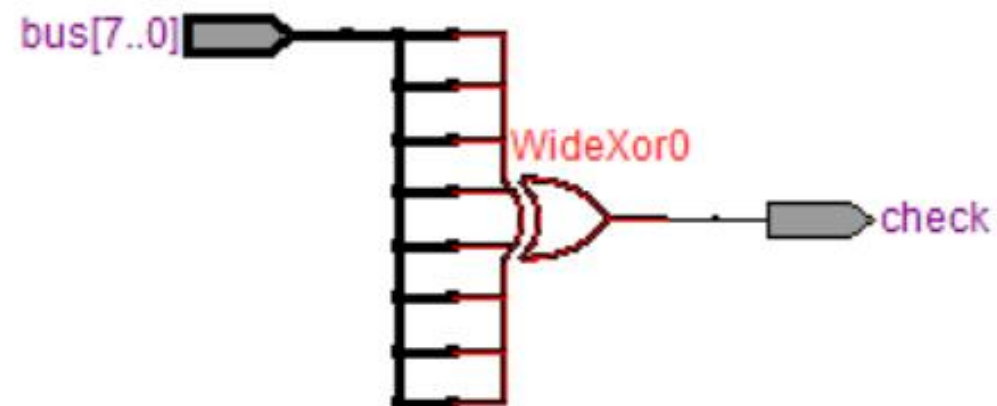
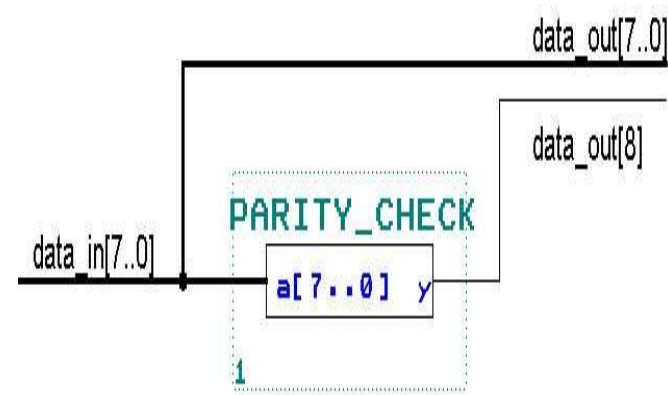


5. 8位奇偶校验位产生电路

```
module odd_che (  
    input [7:0] bus ,  
    output      check  
);  
  
    assign check = ^bus ;  
  
endmodule
```

奇偶？

```
assign check = ~ ^bus ;
```



二、 时序逻辑电路设计

锁存器、寄存器、计数器、分频器、节拍发生器、状态机等。

时钟上升沿:

always @ (posedge clk)

时钟下降沿:

always @ (negedge clk)

原则1：边沿触发信号,不能在always过程中再次出现；

原则2：电平触发信号（或异步控制信号），在always过程中必须用if条件语句对敏感信号表中的信号有匹配的表述，明示信号的逻辑行为。

原则3：同步控制信号，不能出现在敏感信号表中；

原则4：敏感信号列表中不能同时出现边沿触发和电平触发；

原则5：异步敏感信号，异步控制信号在敏感信号表里是边沿敏感信号，但电路性能上是电平敏感；

原则6：尽量采用同步时序。

```

module dff (
    input    clk ,
    input    d   ,
    input    reset ,
    output   q   ,
    output   q_not
);

```

```

    reg qtemp ;

```

```

    always @ ( posedge clk ) begin

```

```

        if ( !reset )

```

```

            qtemp <= 1'b0 ;

```

```

        else

```

```

            qtemp <= d ;

```

```

    end

```

```

    assign q = qtemp ;

```

```

    assign q_not = ~qtemp ;

```

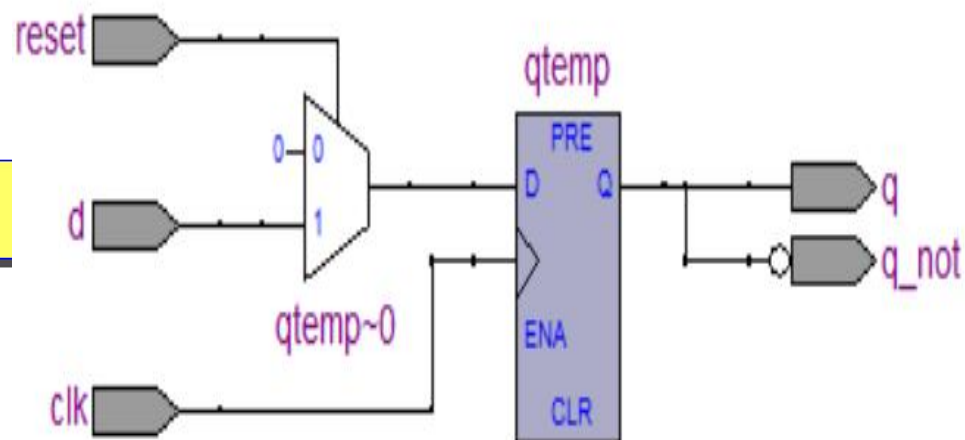
```

endmodule

```

1. 同步复位D触发器(寄存器)

敏感信号表中只有时钟信号
——同步复位。

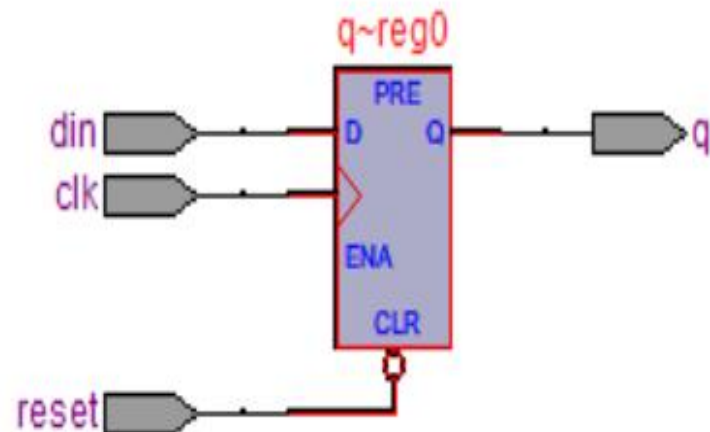


先检查同步信号

if ...else 都对应时钟沿

2. 异步复位D触发器

敏感信号表中除时钟外，还有其它信号——异步操作



```
always @ ( posedge clk , negedge reset ) begin
```

```
    if ( !reset )
```

```
        q <= 1'b0;
```

```
    else
```

```
        q <= din ;
```

```
end
```

先if描述异步

else分支对应时钟沿

- 异步信号必须放在敏感信号表中；
- 必须都是边沿；

同步、异步区别：敏感信号表

3. 带复位的锁存器

```
module latch(
```

```
    input      en    ,
```

```
    input      d      ,
```

```
    input      reset  ,
```

```
    output reg  q
```

```
);
```

```
always @ ( en , d , reset ) begin
```

```
    if ( !reset )
```

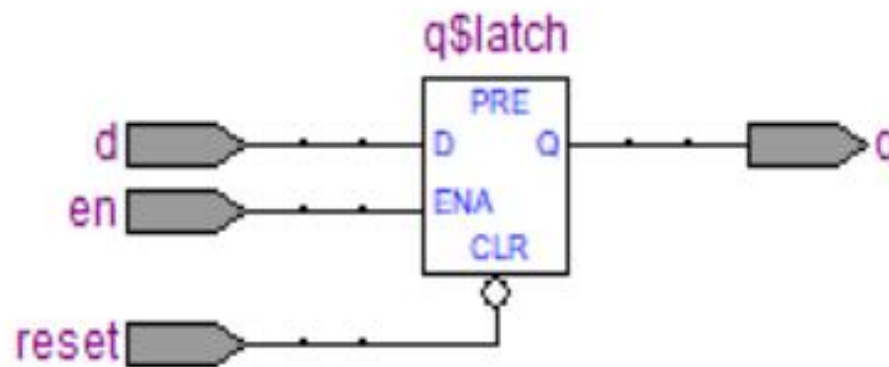
```
        q <= 1'b0 ;
```

```
    else if ( en )
```

```
        q <= d ;
```

```
end
```

```
endmodule
```



- 锁存器的所有输入都放在敏感信号表中；
- 锁存器敏感信号都是电平；
- 不建议综合成锁存器

4. 计数器

```
module count10 (
```

```
    input                clk ,
```

```
    input                load ,
```

```
    input [3:0]          din ,
```

```
    output [3:0]        q ,
```

```
    output              co
```

```
);
```

```
    reg [3:0] qtemp ;
```

```
    always @ ( posedge clk ) begin
```

```
        if (!load)
```

```
            qtemp <= din;
```

```
        else if ( qtemp == 4'd9 )
```

```
            qtemp <= 4'd0 ;
```

```
        else
```

```
            qtemp <= qtemp+1'b1 ;
```

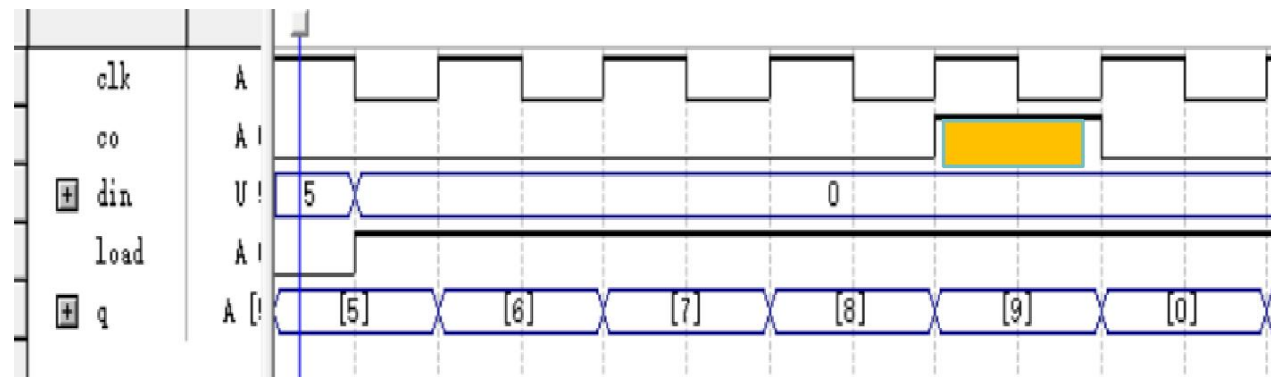
```
    end
```

例：同步置数(低有效) 的10进制计数器。

```
assign q = qtemp ;
```

```
assign co = (qtemp == 4'd9) ;
```

```
endmodule
```



```
always ( posedge clk ) begin
```

```
if ( qtemp == n'dm-1 )
```

```
qtemp <= n'd0 ;
```

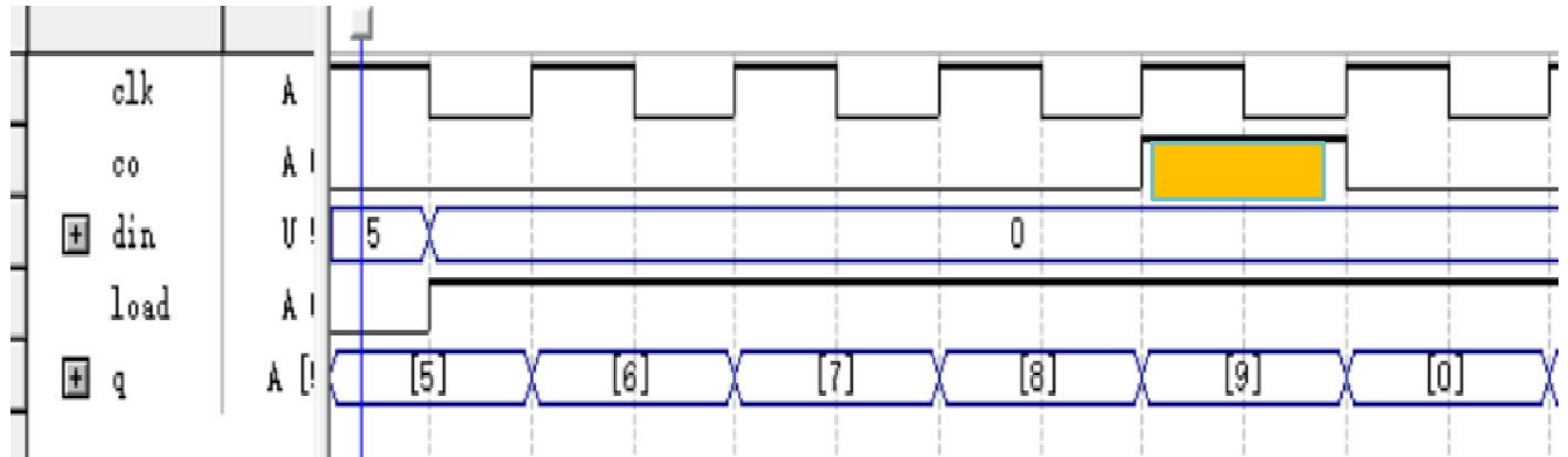
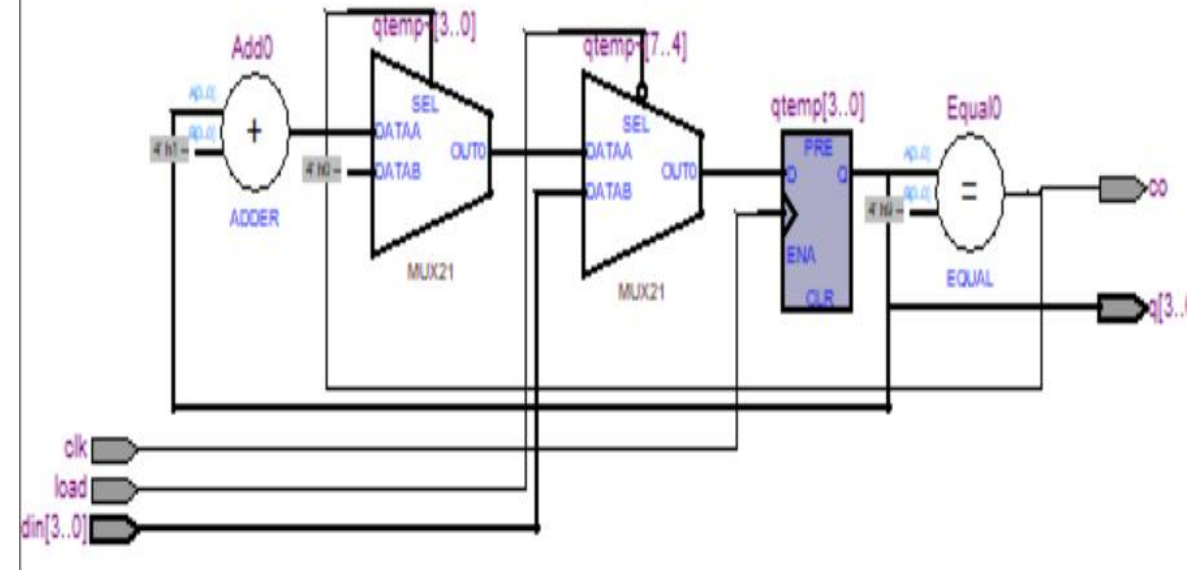
else

```
qtemp <= qtemp+1'b1;
```

end

```
assign  q = qtemp;
```

```
assign    c = (qtemp == n'dm-1) ;
```



```
module count10 (  
    input          clk ,  
    input          load ,  
    input [3:0]    din  ,  
    output reg [3:0] q   ,  
    output reg    co  
);
```

```
always @ (posedge clk) begin
```

```
    if ( !load )
```

```
        q <= din;
```

```
    else if ( q == 4'd9 ) begin
```

```
        q <= 4'd0 ;
```

```
        co <= 1'b1 ;
```

```
    end else begin
```

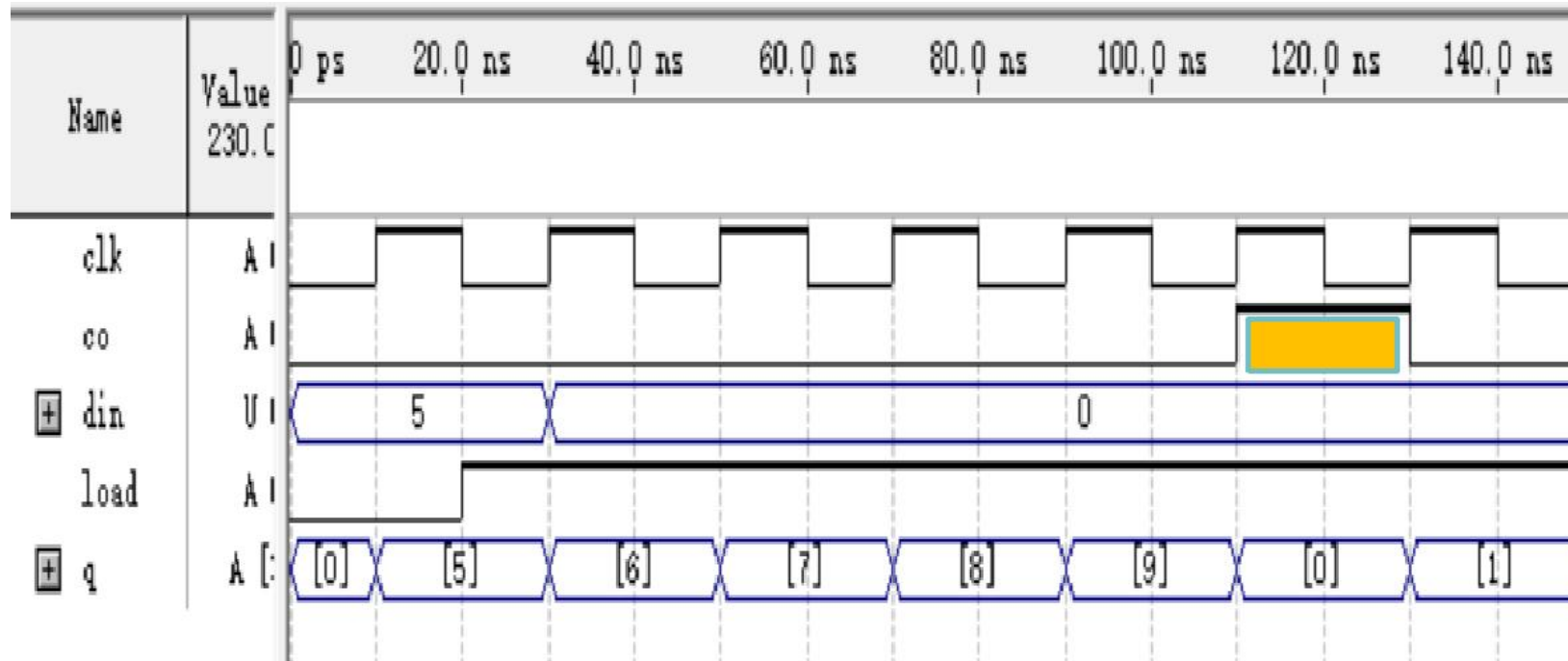
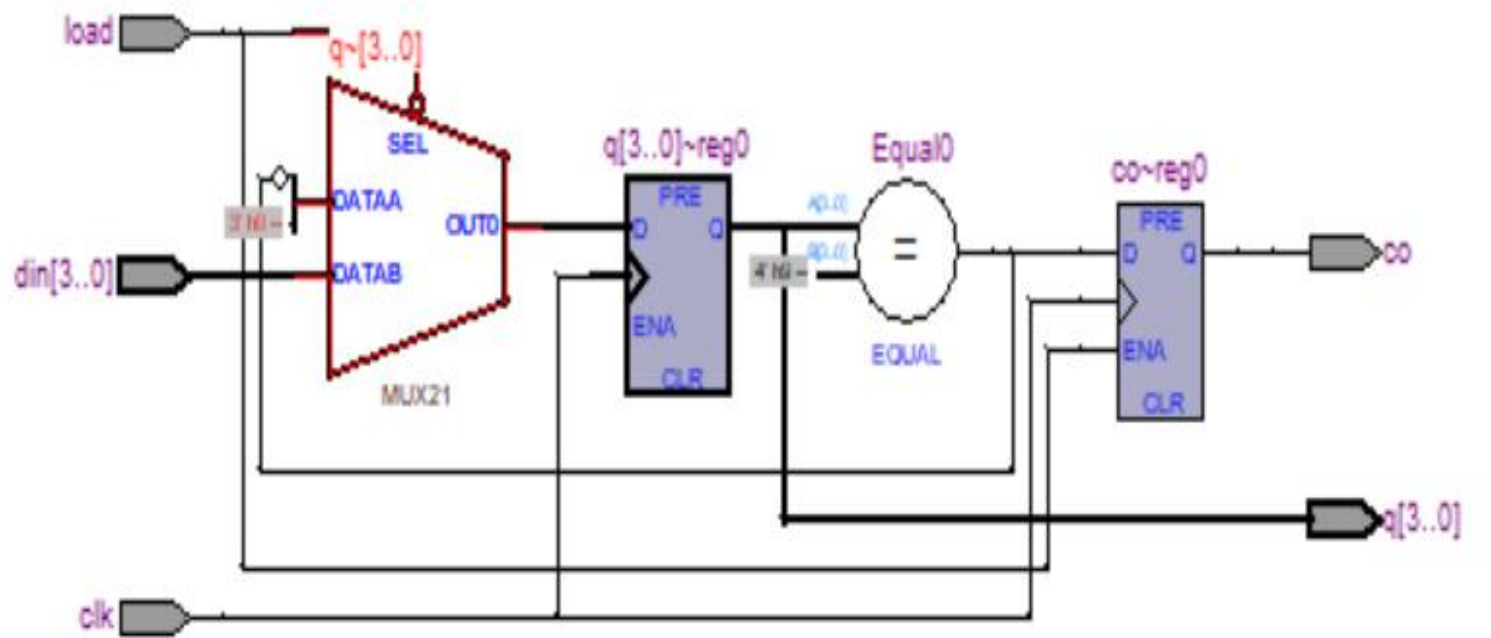
```
        q <= q+1'b1 ;
```

```
        co <= 1'b0 ;
```

```
    end
```

```
end
```

```
endmodule
```



```

module count161 (
    input      clk ,
    input      EP  ,
    input      ET  ,
    input      cr  ,
    input      ld  ,
    input [3:0] din ,
    output reg [3:0] qout ,
    output      co
);

```

例：74LS161。

```

always @ (posedge clk ,  negedge cr ) begin
    if (!cr)
        qout<= 4'b0000 ;
    else if ( !ld )
        qout <= din ;
    else if ( ET && EP )
        qout <= qout + 1'b1 ;
    else
        qout <= qout ;
end

```

```

assign co =  ET & (qout== 4'd15) ;

```

```

endmodule

```

输 入				输 出				CT74LS161
$\overline{CR}$	$\overline{LD}$	EP	ET	$\overline{CP}$	$D_3 D_2 D_1 D_0$	$Q_3 Q_2 Q_1 Q_0$	CO	
0	X	X	X	X	X X X X	0 0 0 0	0	异步置0
1	0	X	X	↑	$d_3 d_2 d_1 d_0$	$d_3 d_2 d_1 d_0$		$CO = E_T \cdot Q_3 Q_2 Q_1 Q_0$
1	1	1	1	↑	X X X X	计 数		$CO = Q_3 Q_2 Q_1 Q_0$
1	1	0	X	X	X X X X	保 持		$CO = E_T \cdot Q_3 Q_2 Q_1 Q_0$
1	1	X	0	X	X X X X	保 持	0	

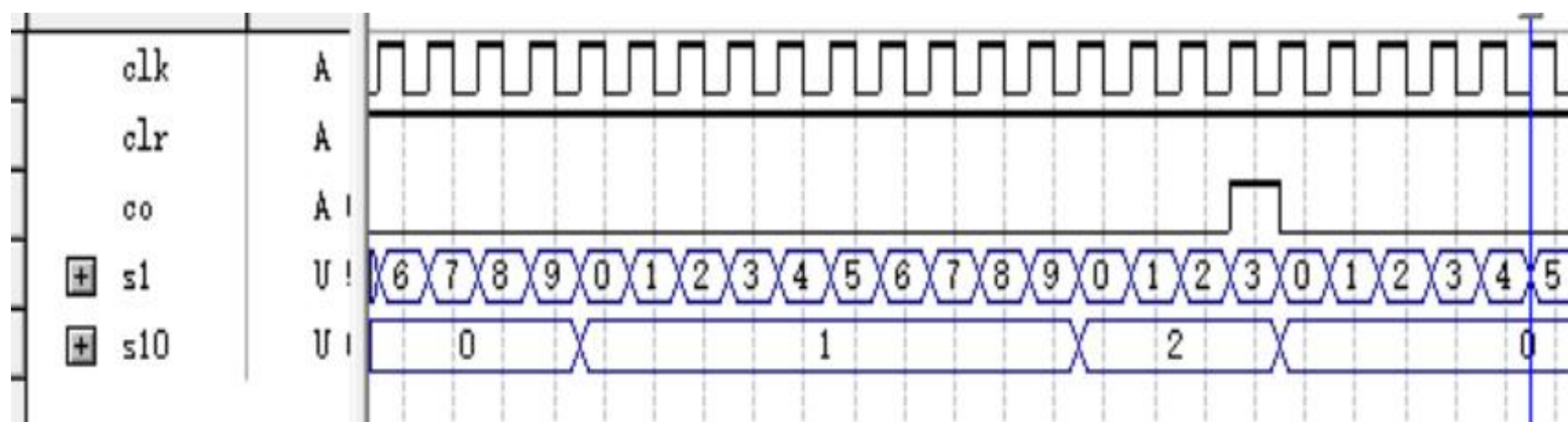
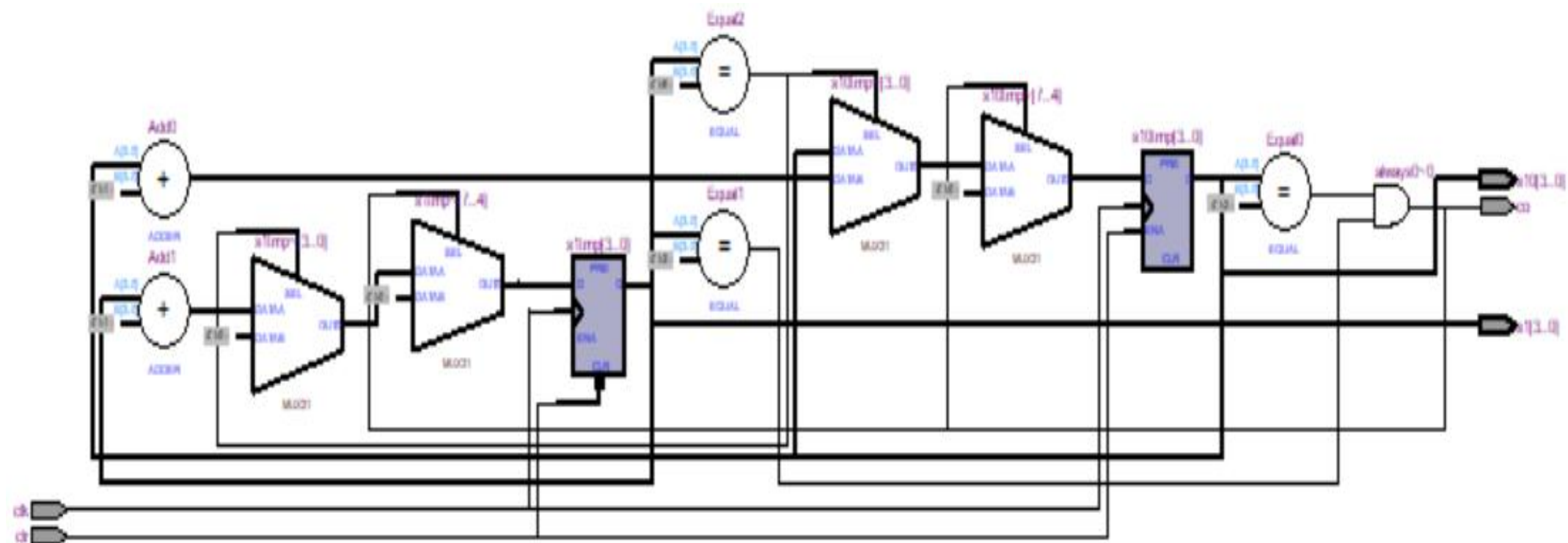
例：异步清零的24进制计数器

与二进制计数器不同，分别做个位、十位计数

```
module count_24 (  
    input      clk ,  
    input      clr  ,  
    output reg [3:0] s1 ,  
    output reg [3:0] s10 ,  
    output co  
);  
  
always @ ( posedge clk ,  negedge clr ) begin  
    if ( !clr ) begin  
        s1 <= 4'd0 ;  
        s10 <= 4'd0 ;  
    end  
end
```

```
    else if ( s10 == 4'd2 && s1 == 4'd3 ) begin  
        s10 <= 4'd0 ;  
        s1  <= 4'd0 ;  
    end else if ( s1 == 4'd9 )  begin  
        s1 <= 4'd0 ;  
        s10 <= s10 + 1'd1 ;  
    end else  
        s1 <= s1 + 1'd1 ;  
    end  
  
assign co = (s10==4'd2) && ( s1==4'd3) ;  
  
endmodule
```





```

module count(
    input                clk ,
    input                x ,
    input                y ,
    output reg [2:0]    q ,
    output              co
);
reg [2:0] m ;

always @ ( * ) begin // 组合过程
    case ({x , y} )
        2'b00:    m = 3'd3 ;
        2'b01:    m = 3'd4 ;
        2'b10:    m = 3'd6 ;
        default: m = 3'd7 ;
    endcase
end

```

模可控计数器（由输入x, y控制）

M=3,4,6,7

```

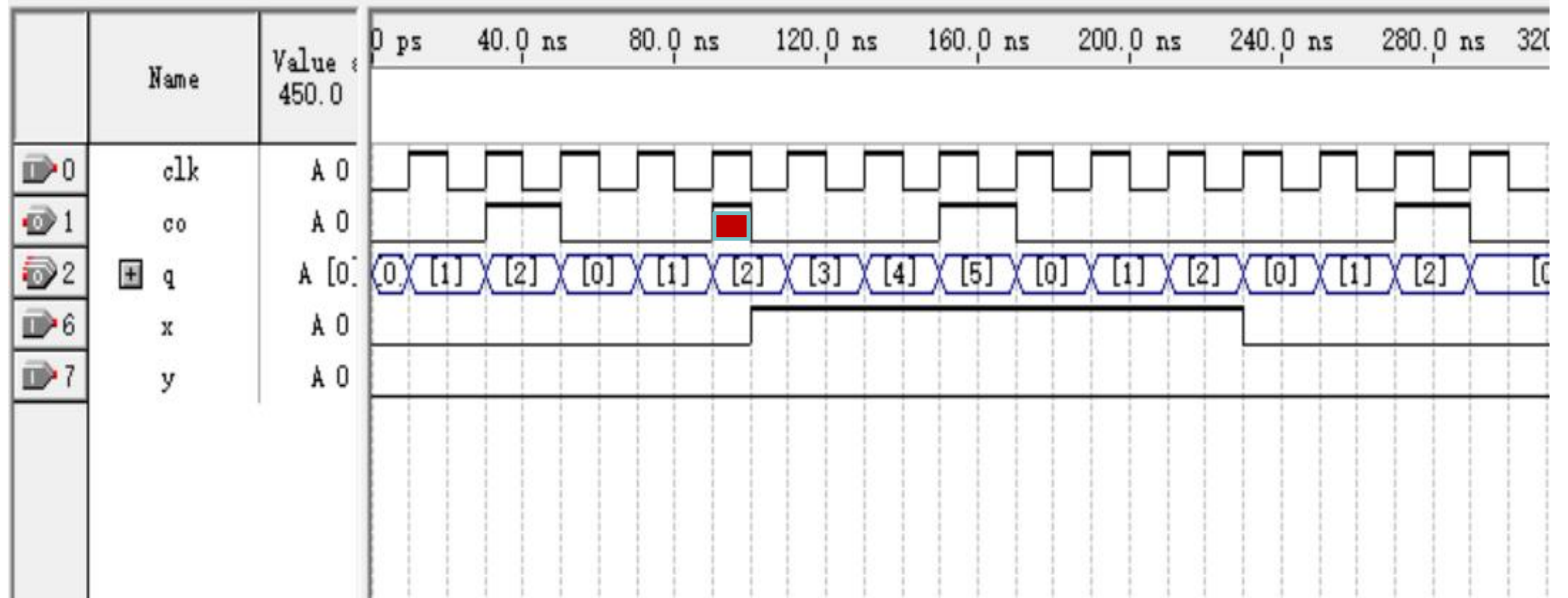
always @ ( posedge clk ) begin
    // 时序过程

    if ( q == m-3'd1 )
        q <= 3'd0 ;
    else
        q <= q + 1'd1 ;
    end

    assign    co = (q == m-3'd1) ;

endmodule

```



```

module divider (
    input clk ,
    output clkout
);
    reg [1:0] count ;
    reg clktemp;

    always @ ( posedge clk ) begin
        if ( count == 2'd2 )
            count <= 2'd0 ;
        else
            count <= count+2'd1 ;
        end

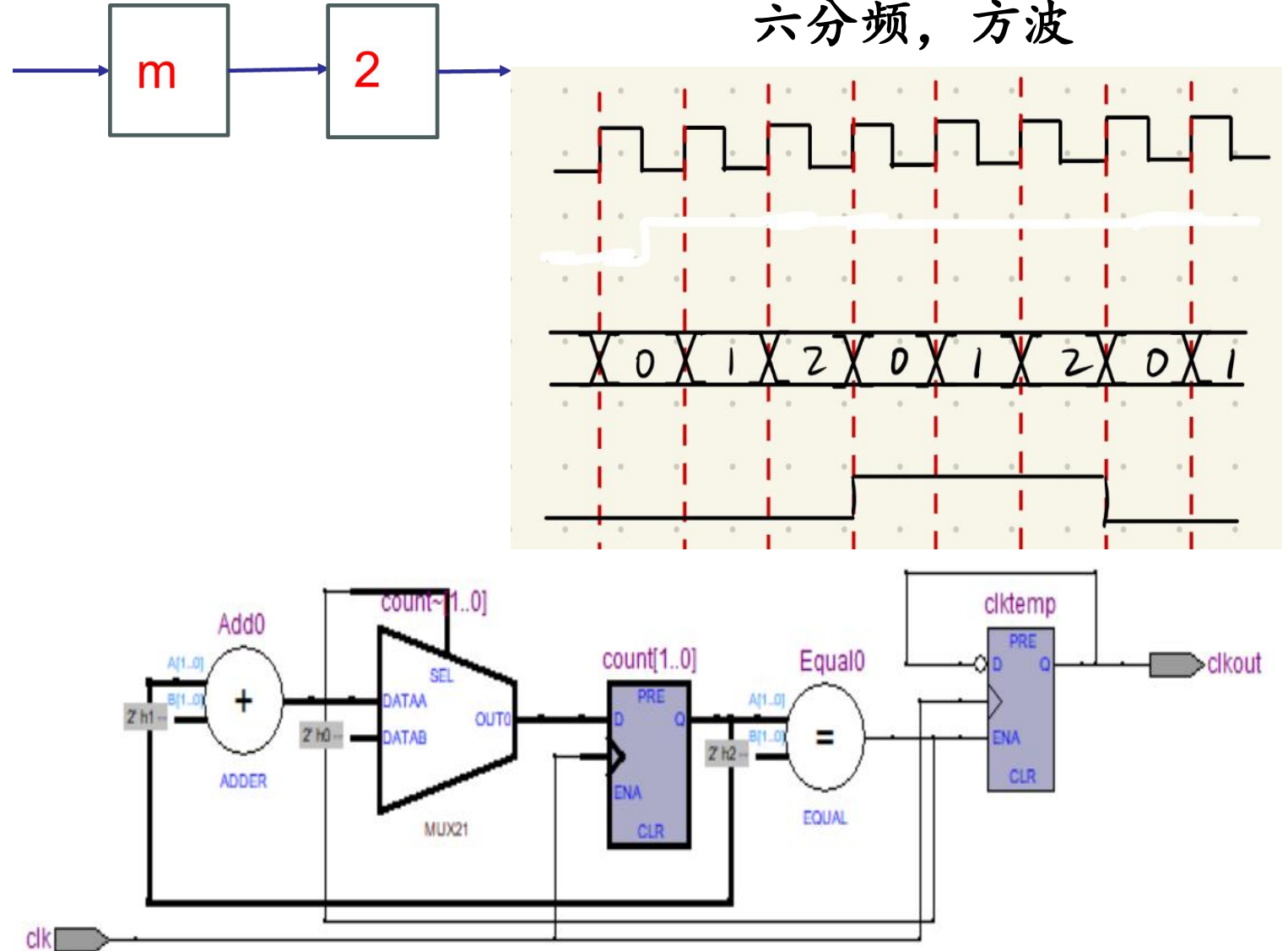
    always @ ( posedge clk ) begin
        if (count == 2'd2 )
            clktemp <= ~clktemp ;
        else
            clktemp <= clktemp;
        end

    assign    clkout = clktemp ;

endmodule

```

5. 设计 $N=2m$ 分频器，占空比50%

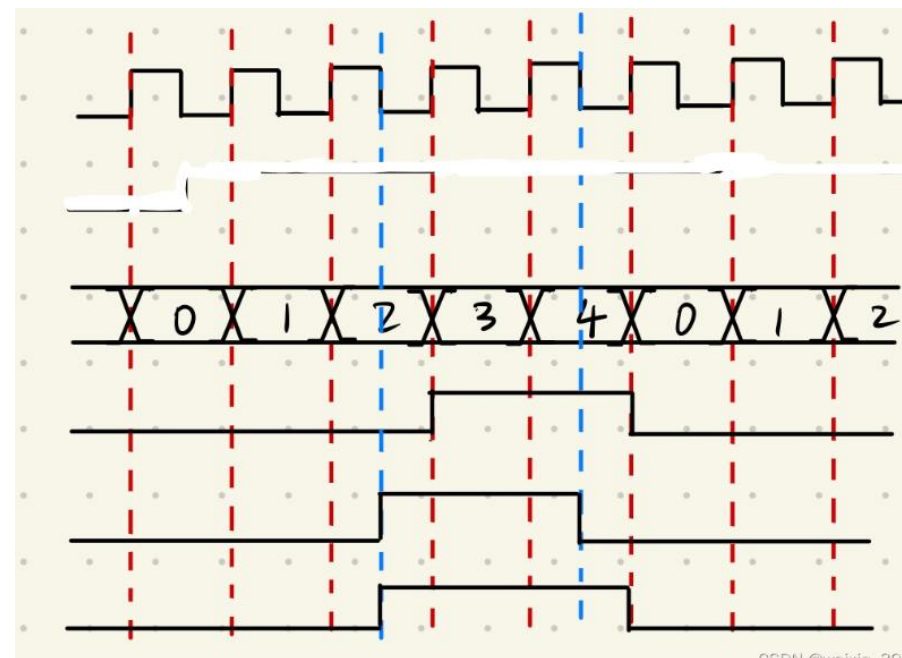


5. 设计 $N=2m-1$ 分频器，占空比50%

双边沿触发：

- 计数器1：模 N 计数器；
- 计数器2：上升沿触发，在计数器1的前 m 状态低电平，后 $N-m$ 个状态高电平；
- 计数器3：下降沿触发，在计数器1的前 m 状态低电平，后 $N-m$ 个状态高电平。
- 输出：计数器2 或 计数器3。

五分频，方波



```

module divider (
    input clk ,
    output clkout
);

```

```

reg [2:0] count1 ;
reg temp2 , temp3;

```

```

always @ ( posedge clk ) begin
    if ( count1 == 3'd4 )
        count1<= 3'd0 ;
    else
        count1 <= count1 + 3'd1 ;
    end

```

```

always @ ( posedge clk ) begin
    if ( count1 == 3'd2 )
        temp2 <= 1'b1 ;
    else if ( count == 3'd4 )
        temp2 <= 1'b0 ;
    else temp2 <= temp2 ;
end

```

```

always @ ( negedge clk ) begin
    if ( count1 == 3'd2 )
        temp3 <= 1'b1 ;
    else if ( count1 == 3'd4 )
        temp3 <= 1'b0 ;
    else temp3 <= temp3;
end

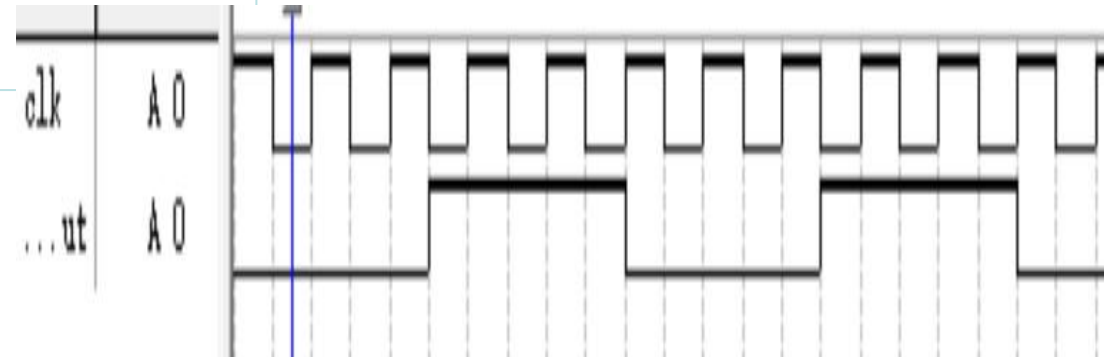
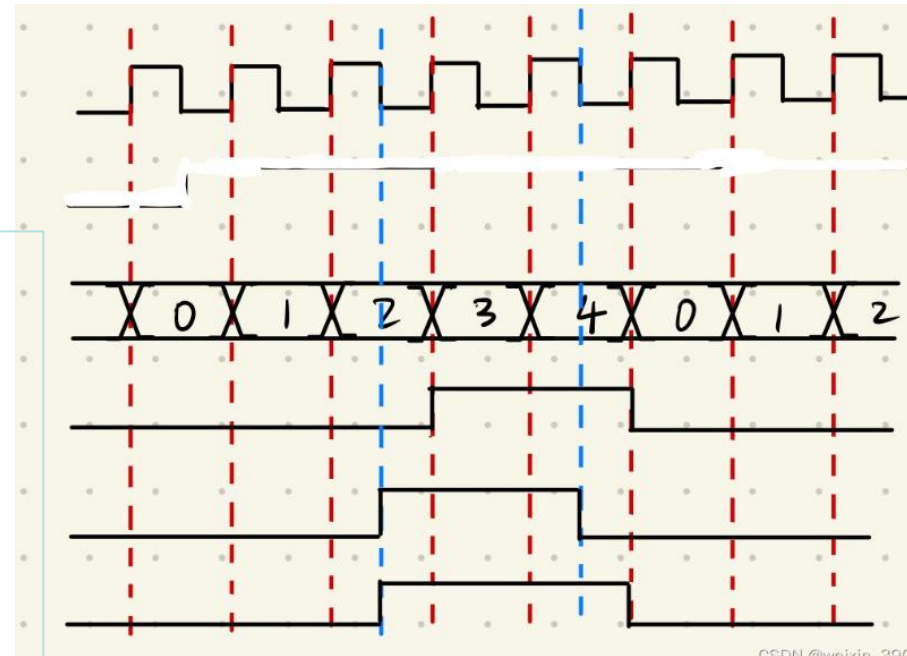
```

```

assign clkout = temp2 | temp3 ;

endmodule

```



```

module shift (
    input      clk    ,
    input      con    ,
    input      load   ,
    output [3:0] qout
);
reg [3:0] qtemp;

always @ ( posedge clk ,  negedge load ) begin
    if ( ! load )
        qtemp <= 4'b1000 ;
    else if ( con )      //左移
        qtemp <= { qtemp[2:0] , qtemp[3] } ;
    else                // 右移
        qtemp <= { qtemp[0] , qtemp[3:1] } ;
end

assign  qout = qtemp ;

endmodule

```

6. 移存型计数器

例：4位环形移位寄存器（左、右移、异步置数**1000**）

7. 状态机

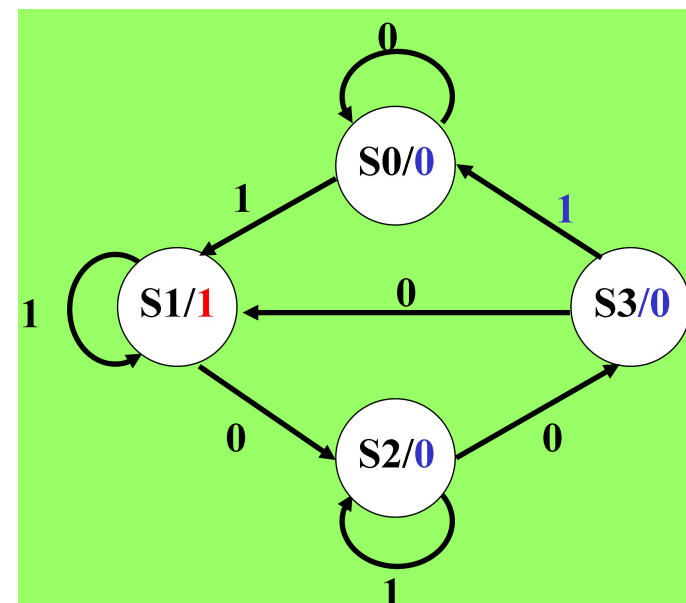
1.) 使用两段式描述:

- 描述时序逻辑: 状态改变, 非阻塞赋值。
- 描述组合逻辑: 次态、输出, 阻塞赋值。

2) 状态机说明语句: 使用parameter给状态编码

3) 状态机要复位: 避免进入挂起状态。

4) 定义两个变量: 现态、次态。



状态机说明： parameter : 各状态编码

```
parameter s0 = 2'b00 ;  
parameter s1 = 2'b01 ;  
parameter s2 = 2'b10 ;  
parameter s3 = 2'b11 ;
```

//二进制码

```
parameter s0 = 2'b00 ;  
parameter s1 = 2'b01 ;  
parameter s2 = 2'b11 ;  
parameter s3 = 2'b10 ;
```

//格雷码

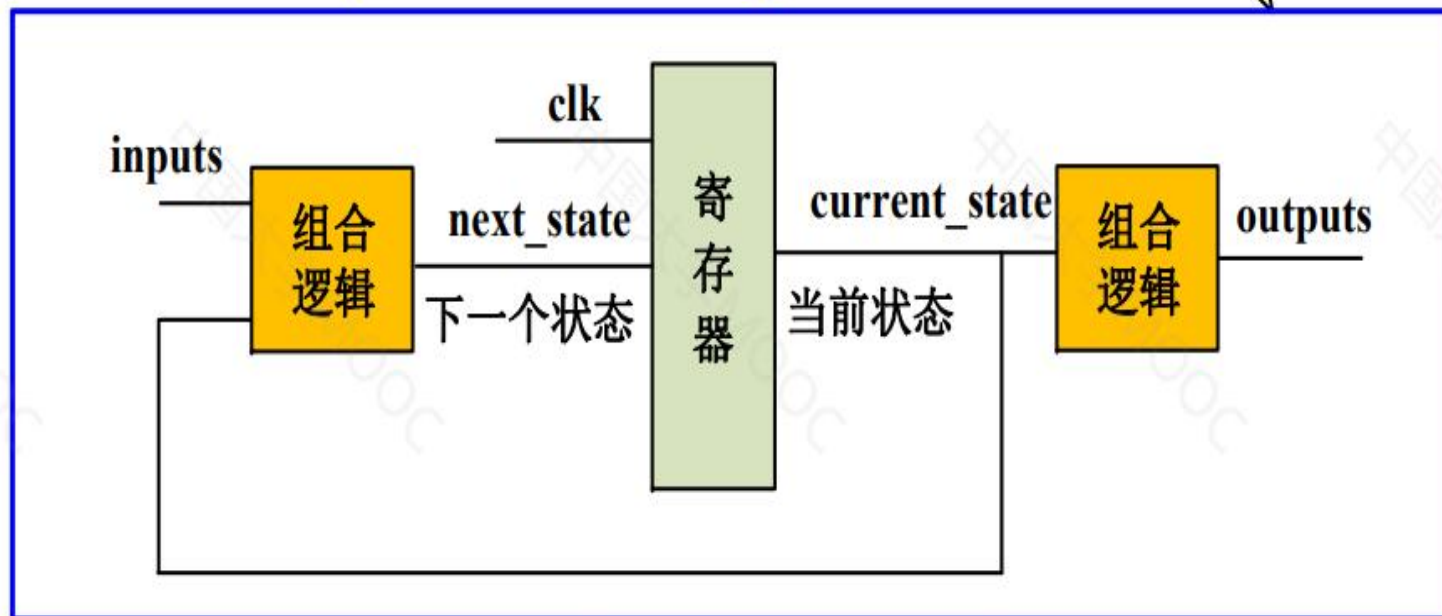
```
parameter s0 = 4'b0001 ;  
parameter s1 = 4'b0010 ;  
parameter s2 = 4'b0100 ;  
parameter s3 = 4'b1000 ;
```

//独热码 (one-hot)

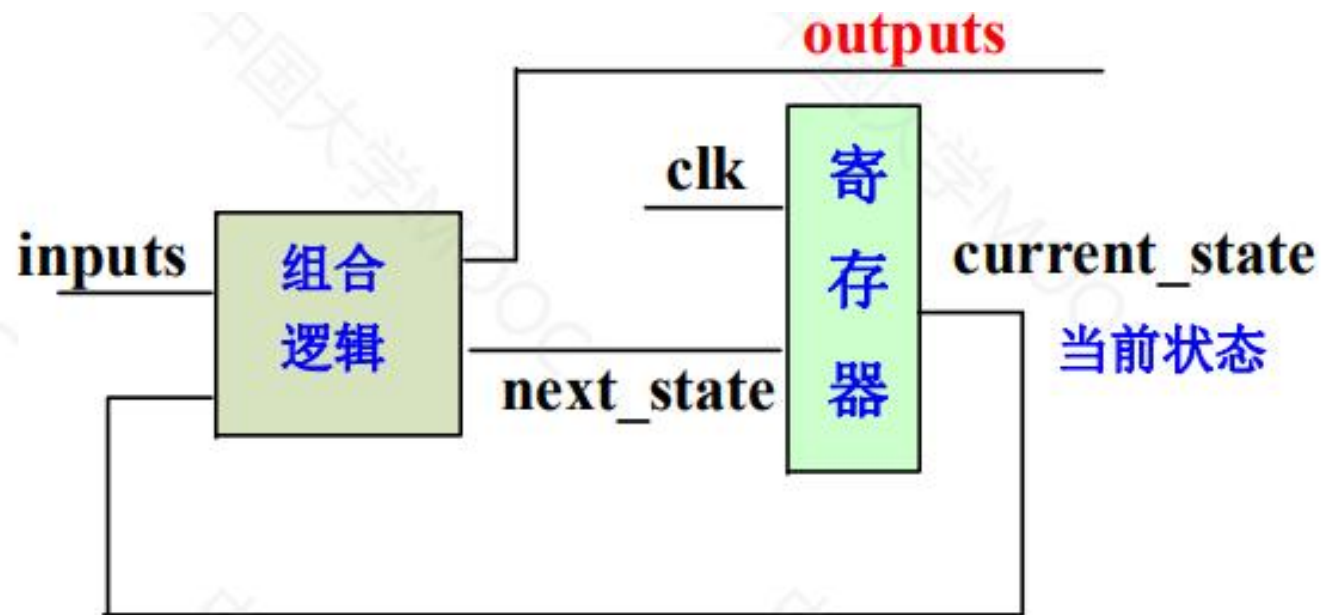
现态、次态定义：

```
reg [3:0] current_state , next_state ; //如用独热码编码
```

Moore 型状态机:



Mealy 型状态机:



Moore 型状态机的描述

```
module moore_fsm # (  
  parameter s0=2'b00 ,  
  parameter s1=2'b01 ,  
  parameter s2=2'b11 ,  
  parameter s3=2'b10  
) (  
  input      clk  ,  
  input      x    ,  
  input      reset ,  
  output reg y  
);
```

```
reg [1:0] current_state , next_state ;
```

```
always @ ( posedge clk , negedge reset ) begin
```

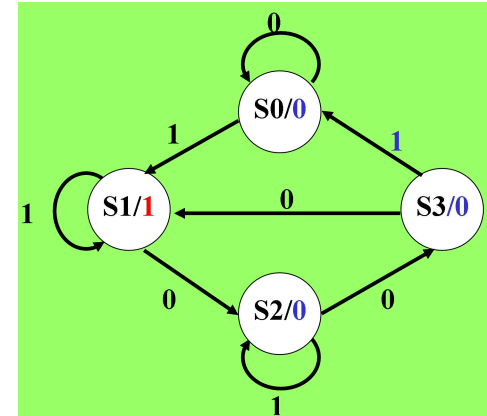
```
  if ( ! reset )
```

```
    current_state <= s0 ;
```

```
  else
```

```
    current_state <= next_state ;
```

```
end
```



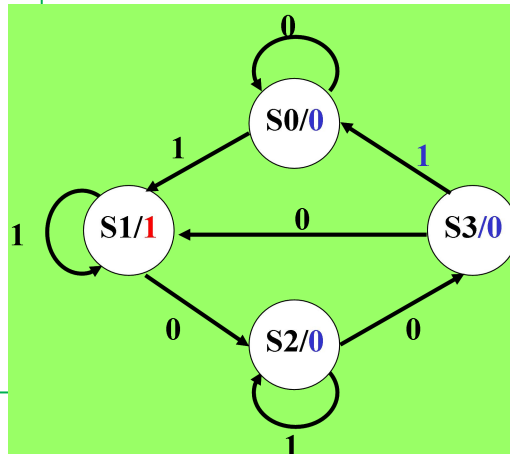
第一进程，完成状态转换，
必须饱含时钟敏感信号

```

always @ ( * ) begin
  case ( current_state )
    s0: begin
      y = 1'b0 ;
      if ( x == 1'b0 )
        next_state = s0 ;
      else
        next_state = s1 ;
      end
    s1: begin
      y = 1'b1 ;
      if ( x == 1'b0 )
        next_state = s2 ;
      else
        next_state = s1 ;
      end
  endcase
end

```

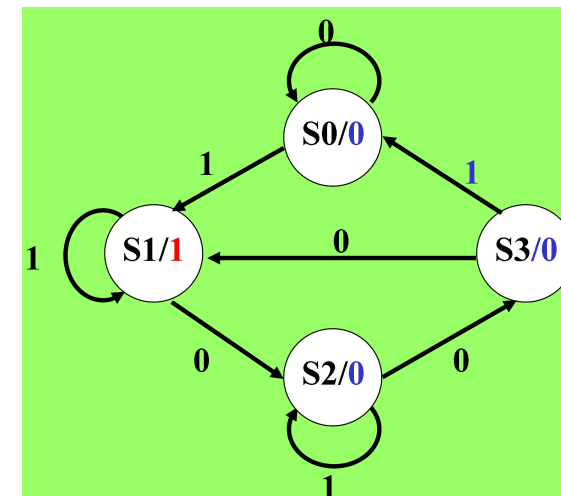
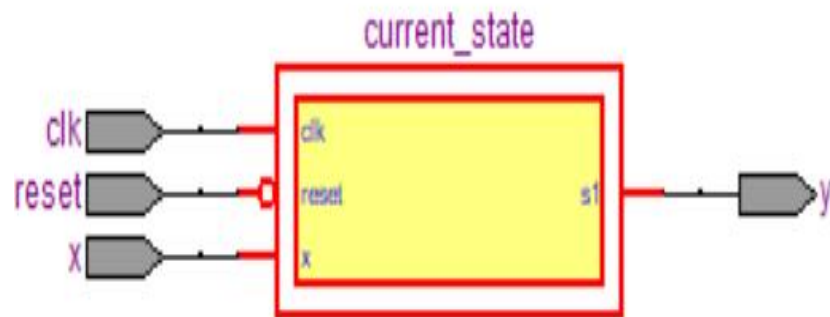
第二进程：组合逻辑，描述次态、输出



```

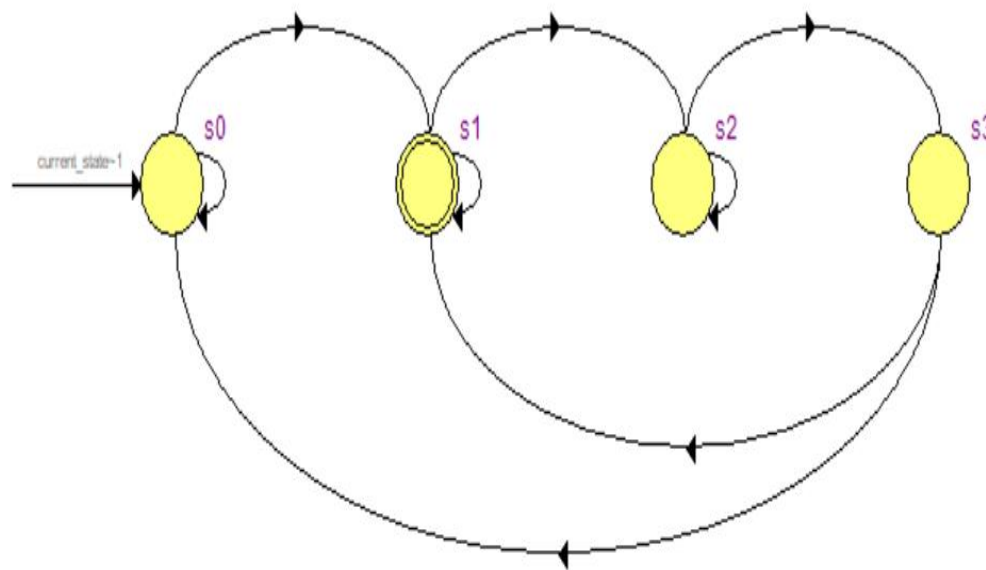
s2: begin
  y = 1'b0 ;
  if ( x == 1'b0 )
    next_state = s3 ;
  else
    next_state = s2 ;
  end
s3: begin
  y = 1'b0 ;
  if ( x == 1'b0 )
    next_state = s1 ;
  else
    next_state = s0 ;
  end
default: begin
  y = 1'b0 ;
  next_state = s0 ;
  end
endcase
end
endmodule

```



编译之后，点击

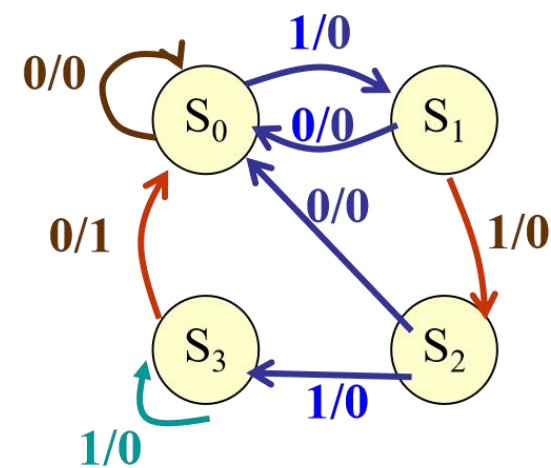
Tool>>Netlist Viewers>>State Machine Viewer



Mealy 型有限状态机的设计

```
module mealy_fsm # (  
    parameter    s0=2'b00 ,  
    parameter    s1=2'b01 ,  
    parameter    s2=2'b11 ,  
    parameter    s3=2'b10  
    ) (  
        input      clk    ,  
        input      x      ,  
        input      reset ,  
        output reg  y  
    );
```

```
reg [1:0]  current_state ;  
reg [1:0]  next_state  ;
```



```
always @ ( posedge clk or negedge reset ) begin  
    if ( ! reset )  
        current_state <= s0 ;  
    else  
        current_state <= next_state ;  
end
```

```

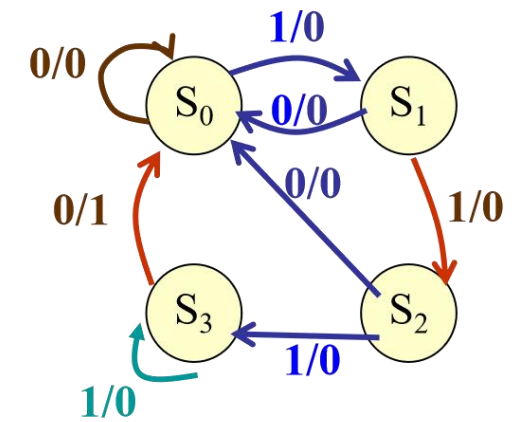
always @ ( * ) begin
  case ( current_state )
    s0: begin
      if ( x == 1'b0 ) begin
        next_state = s0 ;
        y = 1'b0 ;
      end else begin
        next_state = s1 ;
        y = 1'b0 ;
      end
    end
    s1: begin
      if ( x == 1'b0 ) begin
        next_state = s0 ;
        y = 1'b0 ;
      end else begin
        next_state = s2 ;
        y = 1'b0 ;
      end
    end
  end

```

```

s2: begin
  if ( x == 1'b0 ) begin
    next_state = s0 ;
    y = 1'b0 ;
  end else begin
    next_state = s3 ;
    y = 1'b0 ;
  end
end
s3: begin
  if ( x == 1'b0 ) begin
    next_state = s0 ;
    y = 1'b1 ;
  end else begin
    next_state = s3 ;
    y = 1'b0 ;
  end
end

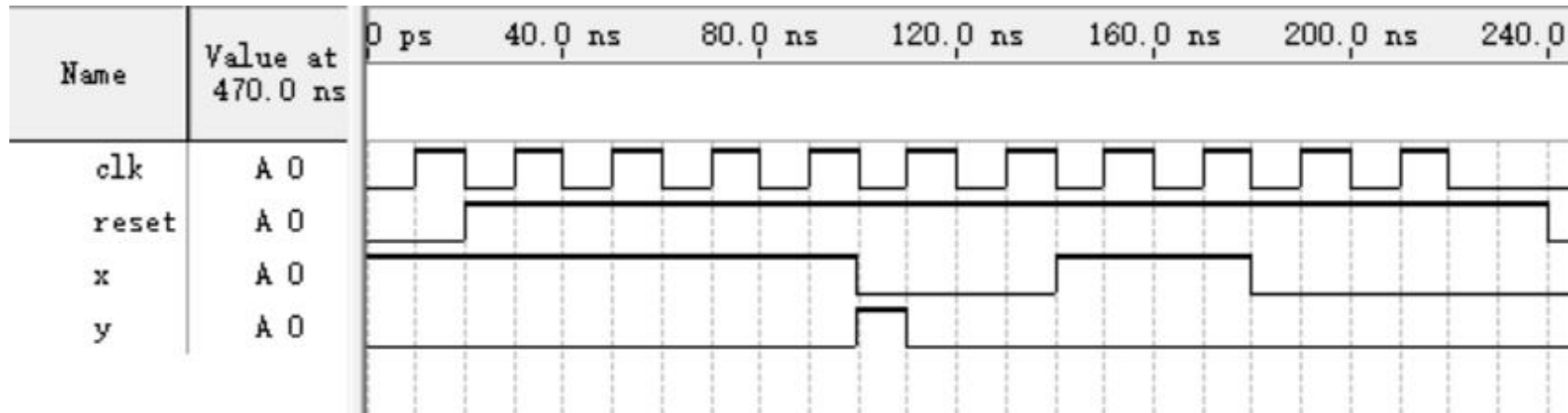
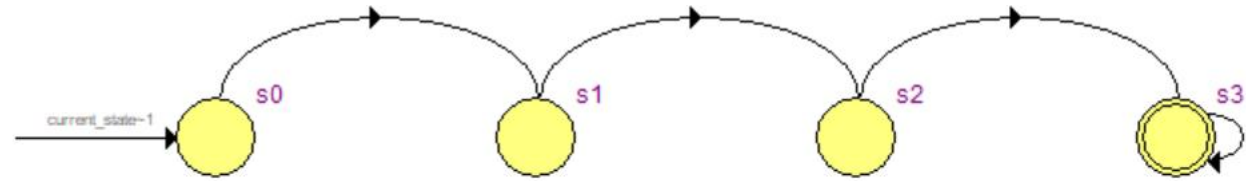
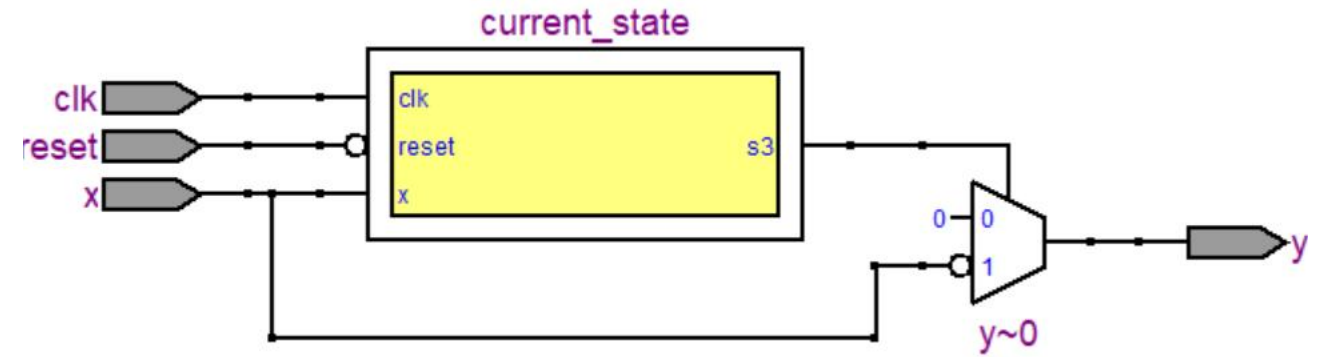
```



```

default: begin
    y = 1'b0 ;
    next_state = s0 ;
end
endcase
end
endmodule

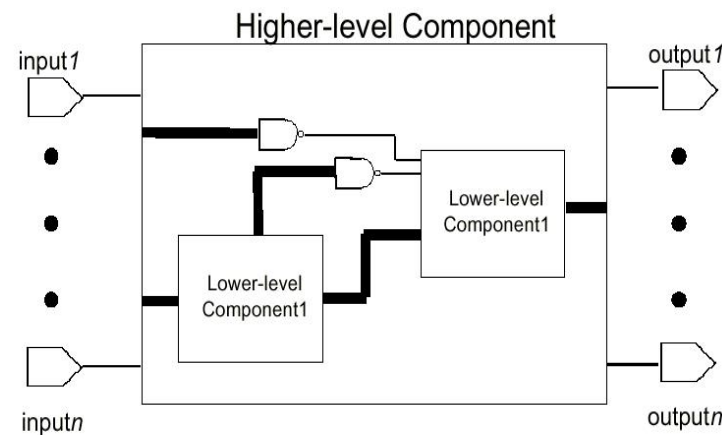
```



5.5.4 verilog的层次化（结构化）设计

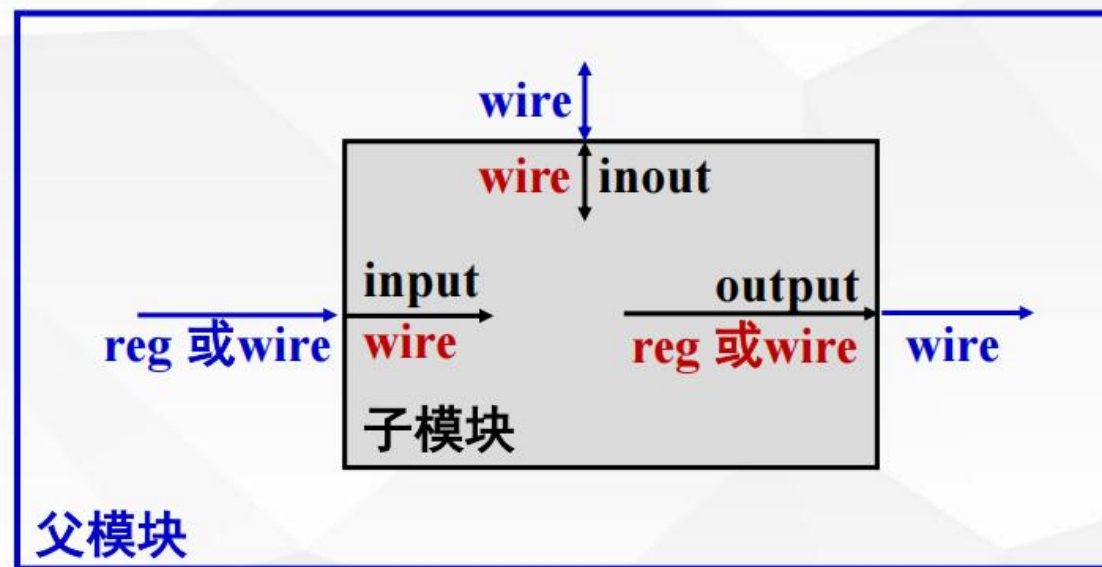
在多层次的设计中，高层次的设计模块调用低层次的设计模块，构成模块化的设计。重点描述模块间的连接关系。

子模块名 #() 例化名 (端口关联表) ;



注意:

- 元件是一段结构完整的 `module` 模块。
- 不能在 `always` 语句内部调用子模块。
- 有关模块端口数据类型的规定



```

module nand_gate (
    input      a      ,
    input      b      ,
    output     y
);

```

```

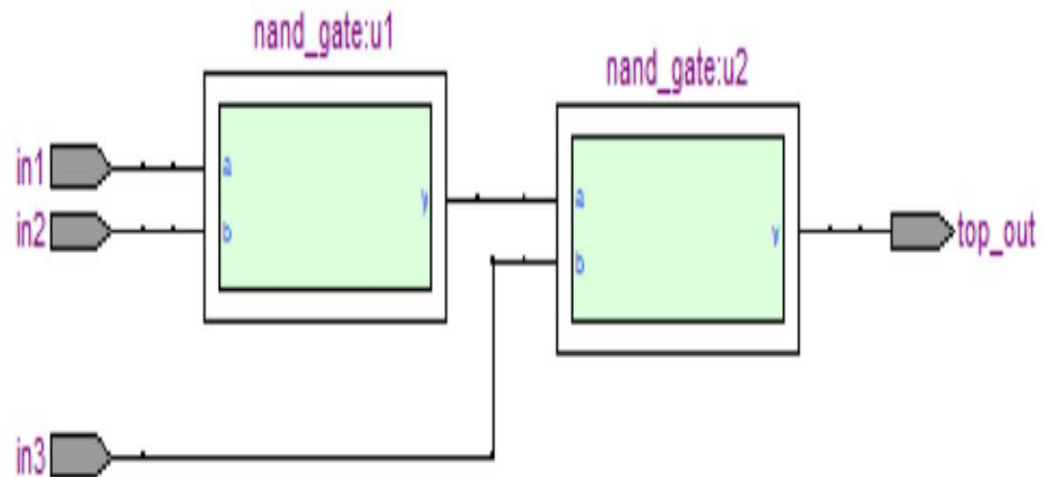
    assign y = ~( a &b );

```

```

endmodule

```



```

module nand_top (
    input      in1      ,
    input      in2      ,
    input      in3      ,
    output     top_out
);

```

```

    wire      c      ;

```

```

    nand_gate u1 (
        .a ( in1 )      ,
        .b ( in2 )      ,
        .y ( c )
    );

```

```

    nand_gate u2 (
        .a ( c )          ,
        .b ( in3 )        ,
        .y ( top_out )
    );

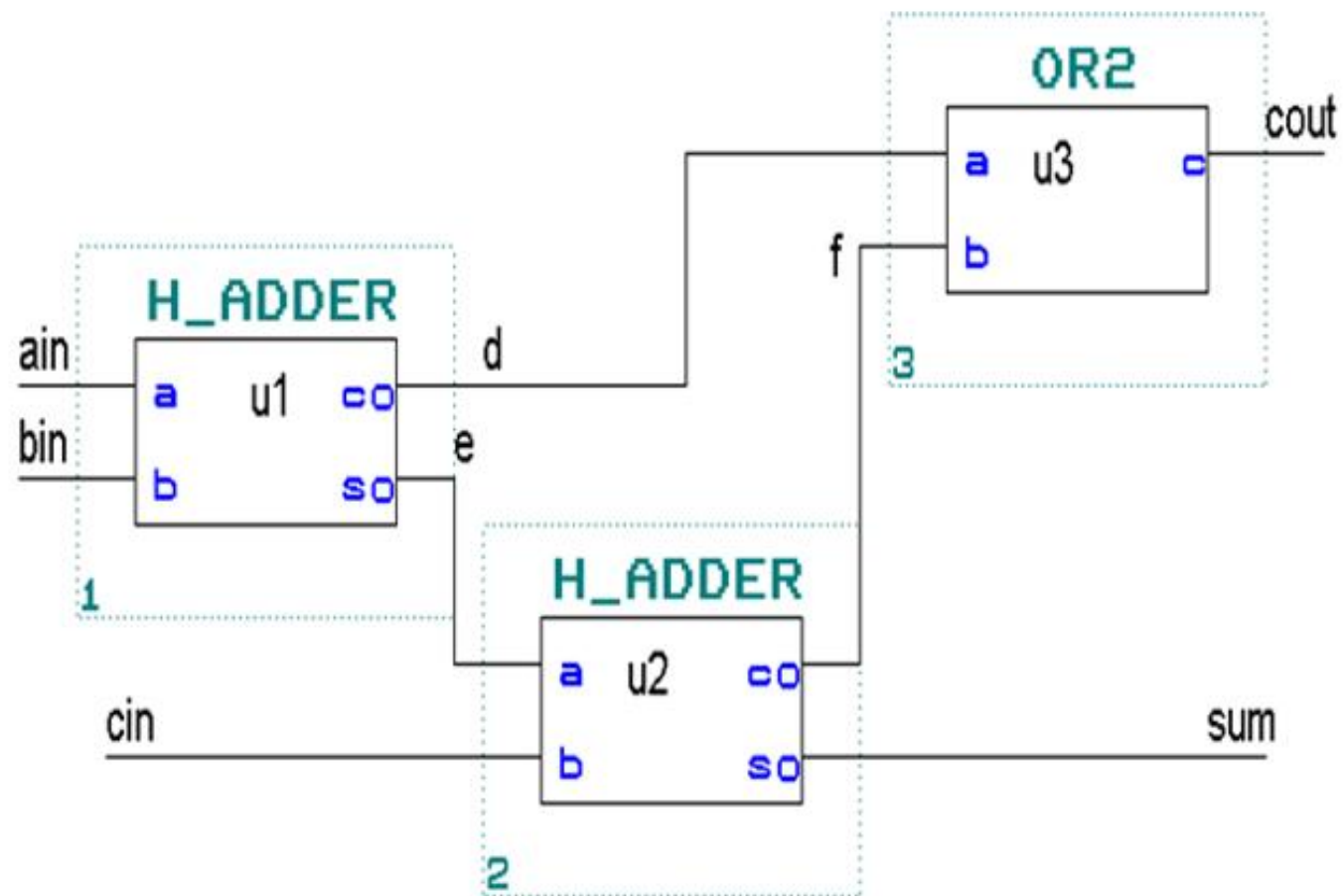
```

```

endmodule

```

一位全加器原理图



```

module h_adder (      //半加器
    input    a    ,
    input    b    ,
    output   so   ,
    output   co   )
;

```

```

assign so = a ^ b ;
assign co = a & b ;

```

```

endmodule

```

```

module or_2 (      //或门
    input    a    ,
    input    b    ,
    output   c
);

```

```

assign c = a | b ;

```

```

endmodule

```

```

module adde(          //顶层逻辑
    input    ain    ,
    input    bin    ,
    input    cin    ,
    output   sum    ,
    output   cout
);

```

```

wire d , e , f ;

```

```

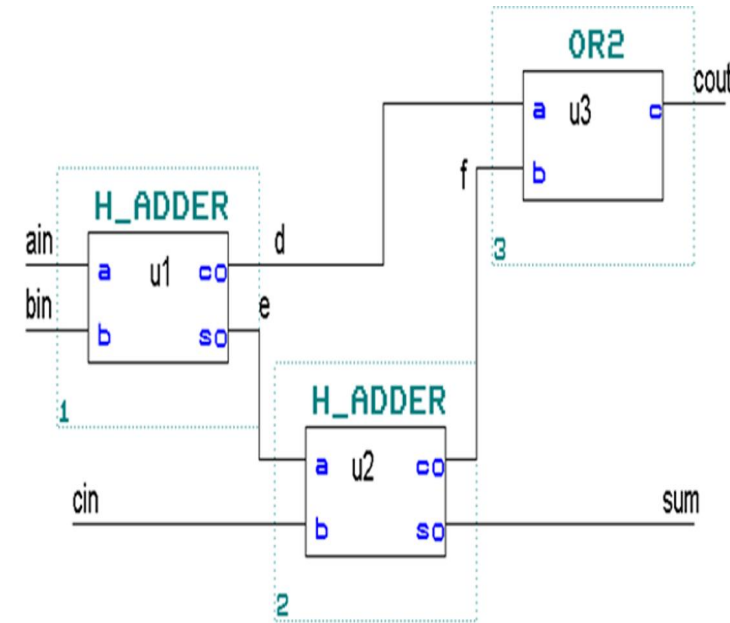
h_adder u1 (
    .a ( ain ) ,
    .b ( bin ) ,
    .so ( e ) ,
    .co ( d )
);

```

```

h_adder u2 (
    .a ( e ) ,
    .b ( cin ) ,
    .so ( sum ) ,
    .co ( f )
);

```



```

or_2 u3 (
    .a ( d ) ,
    .b ( f ) ,
    .c ( cout )

```

```

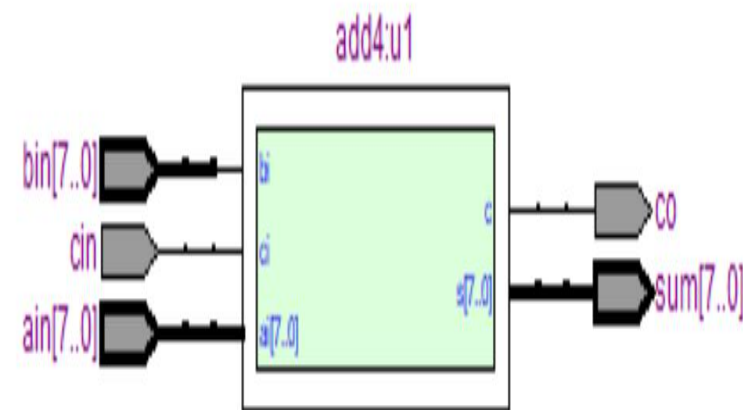
endmodule

```

有参数的例化

```
module add4 # (  
    parameter N = 4  
) (  
    input [N-1:0]    ai ,  
    input [N-1:0]    bi ,  
    input            ci ,  
    output [N-1:0]    s ,  
    output           c  
);  
  
assign { c,s } = ai+bi+ci ;  
  
endmodule
```

```
module adde ( //顶层逻辑  
    input [7:0]    ain ,  
    input [7:0]    bin ,  
    input          cin ,  
    output [7:0]    sum ,  
    output         co  
);  
  
add4 #( .N(8) ) u1 (  
    .ai (ain) ,  
    .bi (bin) ,  
    .ci (cin) ,  
    .s (sum) ,  
    .c (co)  
);  
  
endmodule
```



改参数的例化

```

module lampion (
    input      clk      ,
    input      clr      ,
    output reg [7:0]    y
);
reg [8:0] count_500 ;
reg [4:0] count_32 ;

always @( posedge clk or negedge clr) begin
    if ( !clr )
        count_500 <= 9'd0 ;
    else if ( count_500 == 9'd499 )
        count_500 <= 9'd0 ;
    else
        count_500 <= count_500+1'b1 ;
end

```

每个状态保持时间0.5s；得到
0.5s的节拍信号，计数值500

8个彩灯，4花样自动切换的彩灯控制器，灯亮
时间0.5s：

第1：从右到左，然后从左到右逐次点亮，再
全黑，再全亮；

第2：两边同时亮1个逐次向中间移动再散开；

第3：两边同时亮2个逐次向中间移动再散开；

第4：每三个亮、每四个亮、间隔亮。

共32个种变换，循环往复。

标准时钟1KHz

```

always @( posedge clk or negedge clr) begin
    if ( !clr )
        count_32 <= 5'd0 ;
    else if ( count_500 == 9'd499 )
        count_32 <= count_32+1'b1 ;
end

```

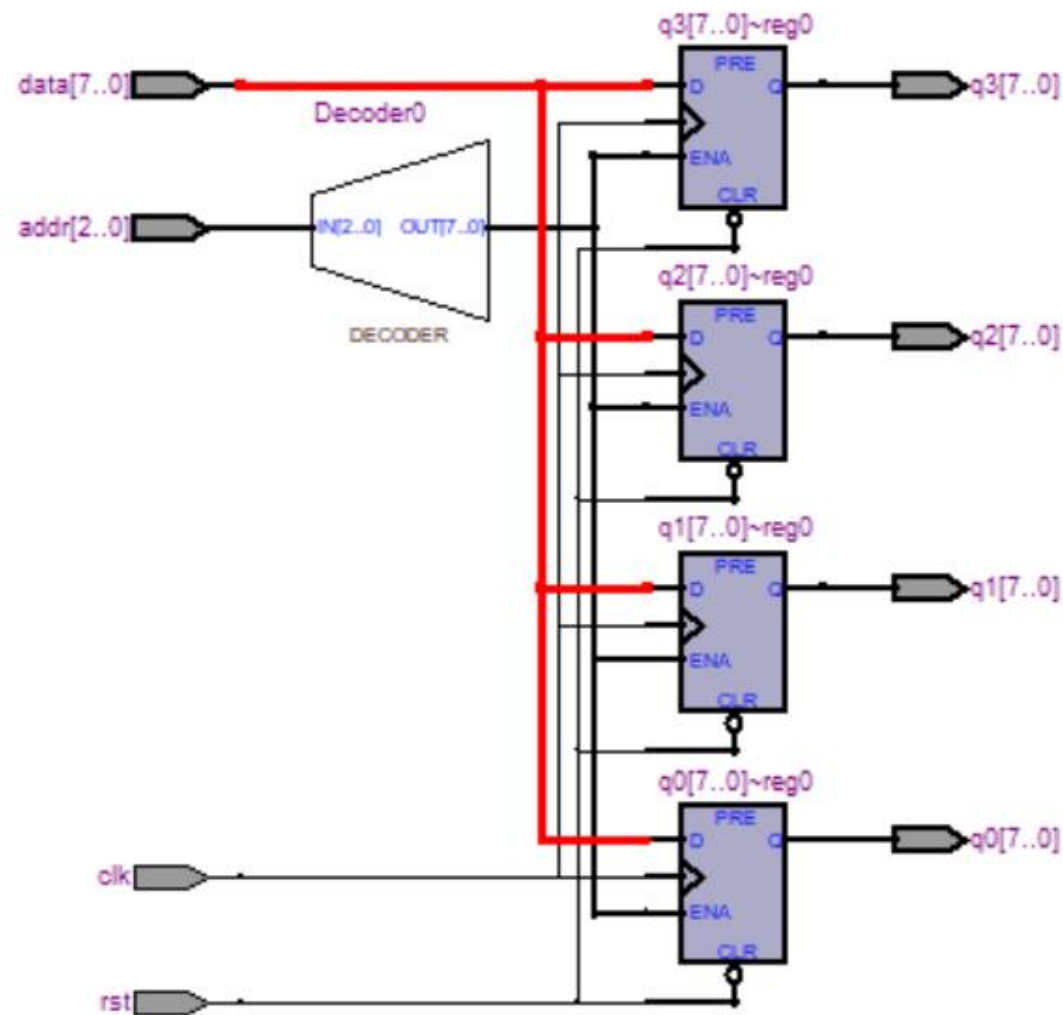
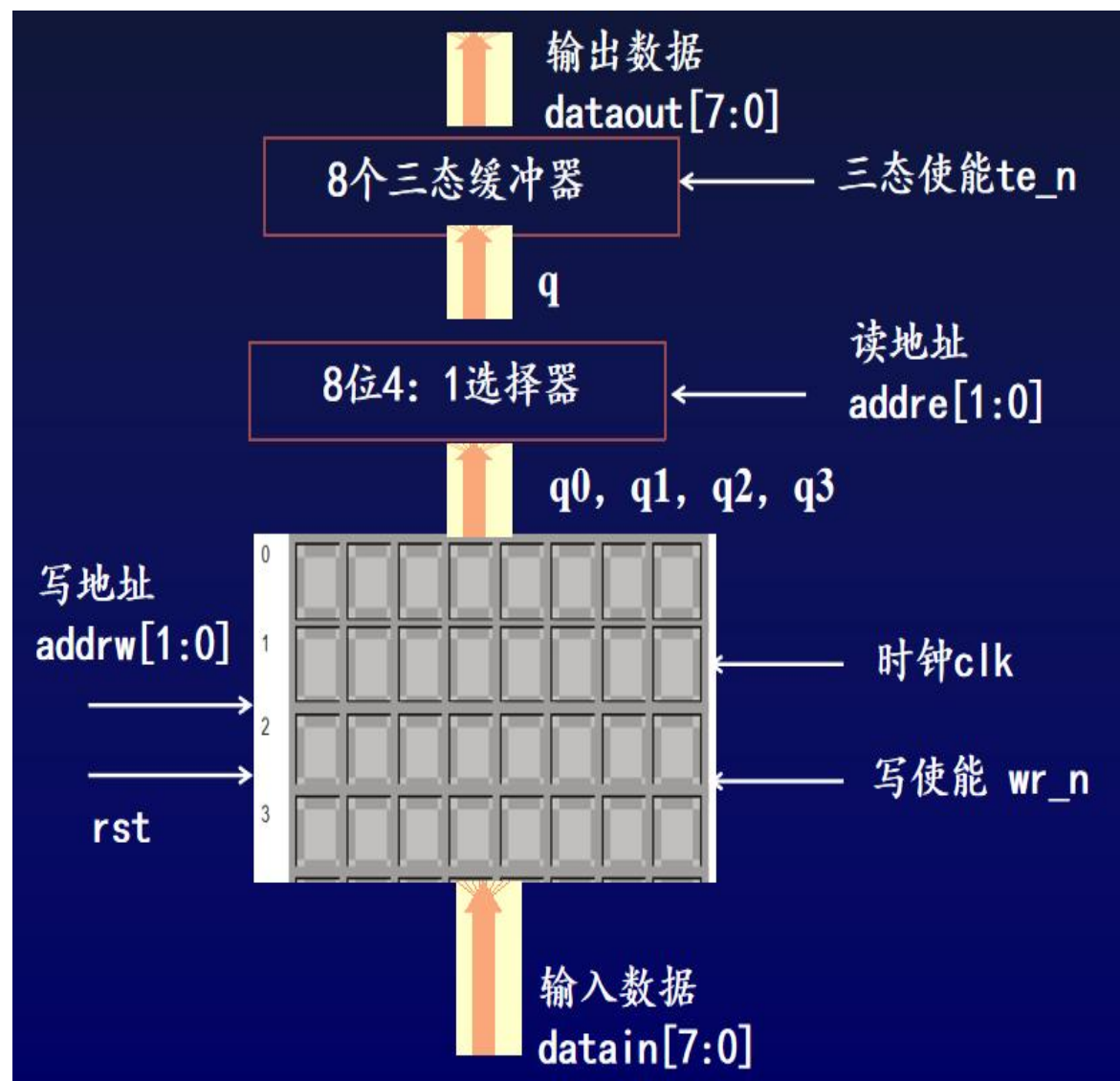
```
always @ ( count_32 ) begin
  case ( count_32 )
    5'd0: y = 8'b0000_0001 ;
    5'd1: y = 8'b0000_0010 ;
    5'd2: y = 8'b0000_0100 ;
    5'd3: y = 8'b0000_1000 ;
    5'd4: y = 8'b0001_0000 ;
    5'd5: y = 8'b0010_0000 ;
    5'd6: y = 8'b0100_0000 ;
    5'd7: y = 8'b1000_0000 ;
    5'd8: y = 8'b1000_0000 ;
    5'd9: y = 8'b0100_0000 ;
    5'd10: y = 8'b0010_0000 ;
    5'd11: y = 8'b0001_0000 ;
    5'd12: y = 8'b0000_1000 ;
    5'd13: y = 8'b0000_0100 ;
    5'd14: y = 8'b0000_0010 ;
    5'd15: y = 8'b0000_0001 ;
    5'd16: y = 8'b0000_0000 ;
    5'd17: y = 8'b1111_1111 ;
    5'd18: y = 8'b1000_0001 ;
```

```
5'd19: y = 8'b0100_0010 ;
5'd20: y = 8'b0010_0100 ;
5'd21: y = 8'b0001_1000 ;
5'd22: y = 8'b0010_0100 ;
5'd23: y = 8'b0100_0010 ;
5'd24: y = 8'b1000_0001 ;
5'd25: y = 8'b1100_0011 ;
5'd26: y = 8'b0011_1100 ;
5'd27: y = 8'b1100_0011 ;
5'd28: y = 8'b1110_0111 ;
5'd29: y = 8'b0111_1110 ;
5'd30: y = 8'b1111_0000 ;
5'd31: y = 8'b1111_0000 ;
    default: y = 8'b0000_0000 ;
  endcase
end

endmodule
```

序列信号发生器如何实现？
顺序脉冲发生器如何实现？

4×8位 寄存器组



```

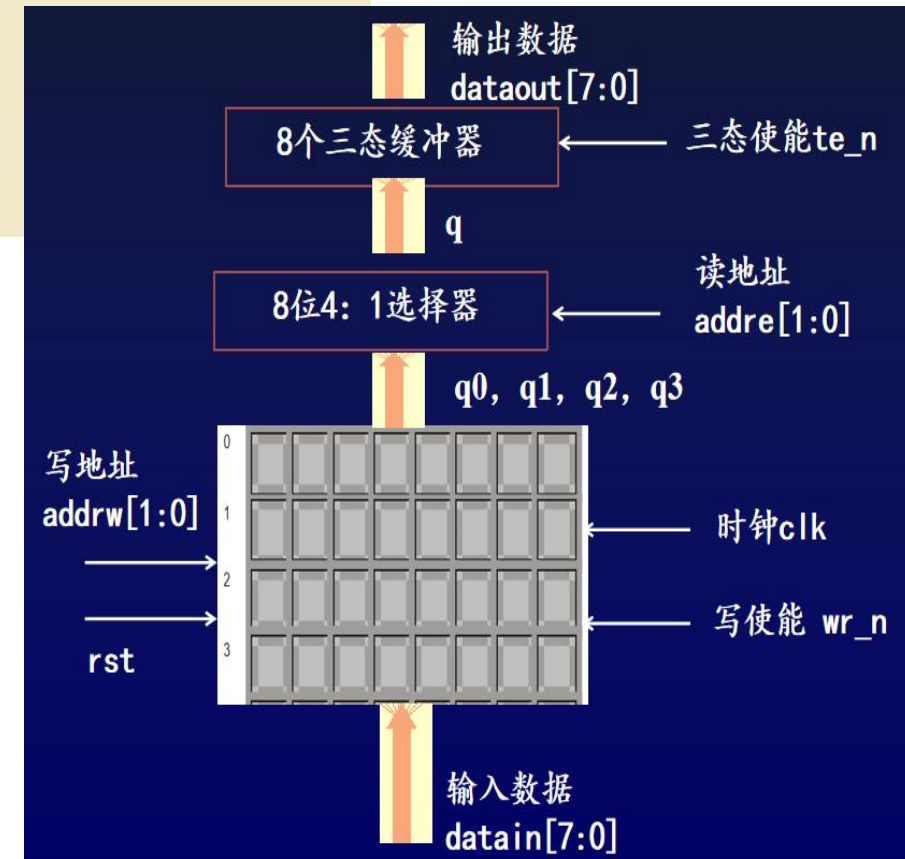
module reg_8 # (    // 寄存器组
    parameter  DW = 8 ,
    parameter  ADW = 2
)
    input                clk      ,
    input                wr_n     ,
    input                rst      ,
    input [ADW-1:0]      addr     ,
    input [DW-1:0]       data     ,
    output reg [DW-1:0]   q0      ,
    output reg [DW-1:0]   q1, q2, q3
);
always @ ( posedge clk , negedge rst) begin
    if ( !rst ) begin
        q0 <= 8'd0 ;
        q1 <= 8'd0 ;
        q2 <= 8'd0 ;
        q3 <= 8'd0 ;
    end else if (!wr_n) begin

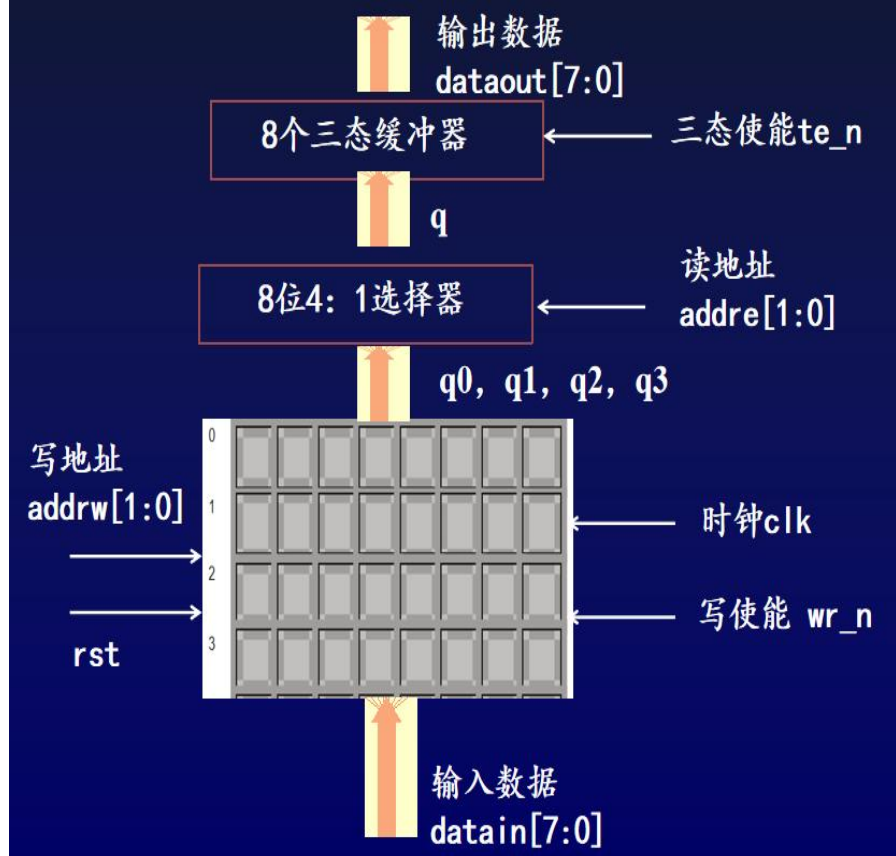
```

```

end else if (!wr_n) begin
    case (addr)
        2'b00: q0 <= data ;
        2'b01: q1 <= data ;
        2'b10: q2 <= data ;
        2'b11: q3 <= data ;
    endcase
end
endmodule

```





```

module mux4 # ( // 8位4:1选择器
    parameter DW=8
) (
    input [1:0]      addr ,
    input [DW-1:0]  d0   ,
    input [DW-1:0]  d1   ,
    input [DW-1:0]  d2   ,
    input [DW-1:0]  d3   ,
    output reg [DW-1:0] f
) ;

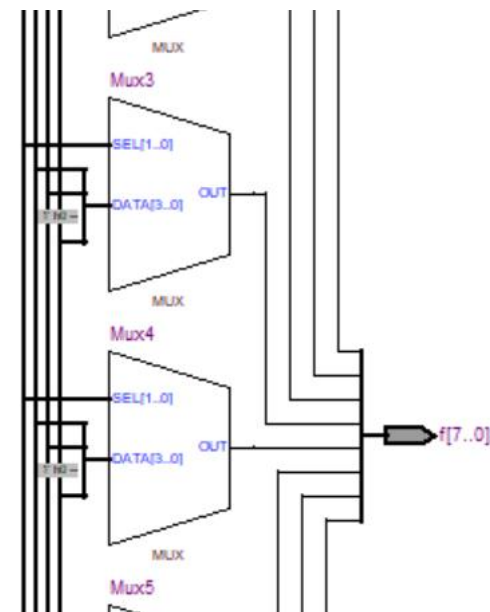
```

```

always @ ( * ) begin
    case ( addr )
        2'b00 : f = d0 ;
        2'b01 : f = d1 ;
        2'b10 : f = d2 ;
        2'b11 : f = d3 ;
        default: f = 8'b1111_1111 ;
    endcase
end

endmodule

```



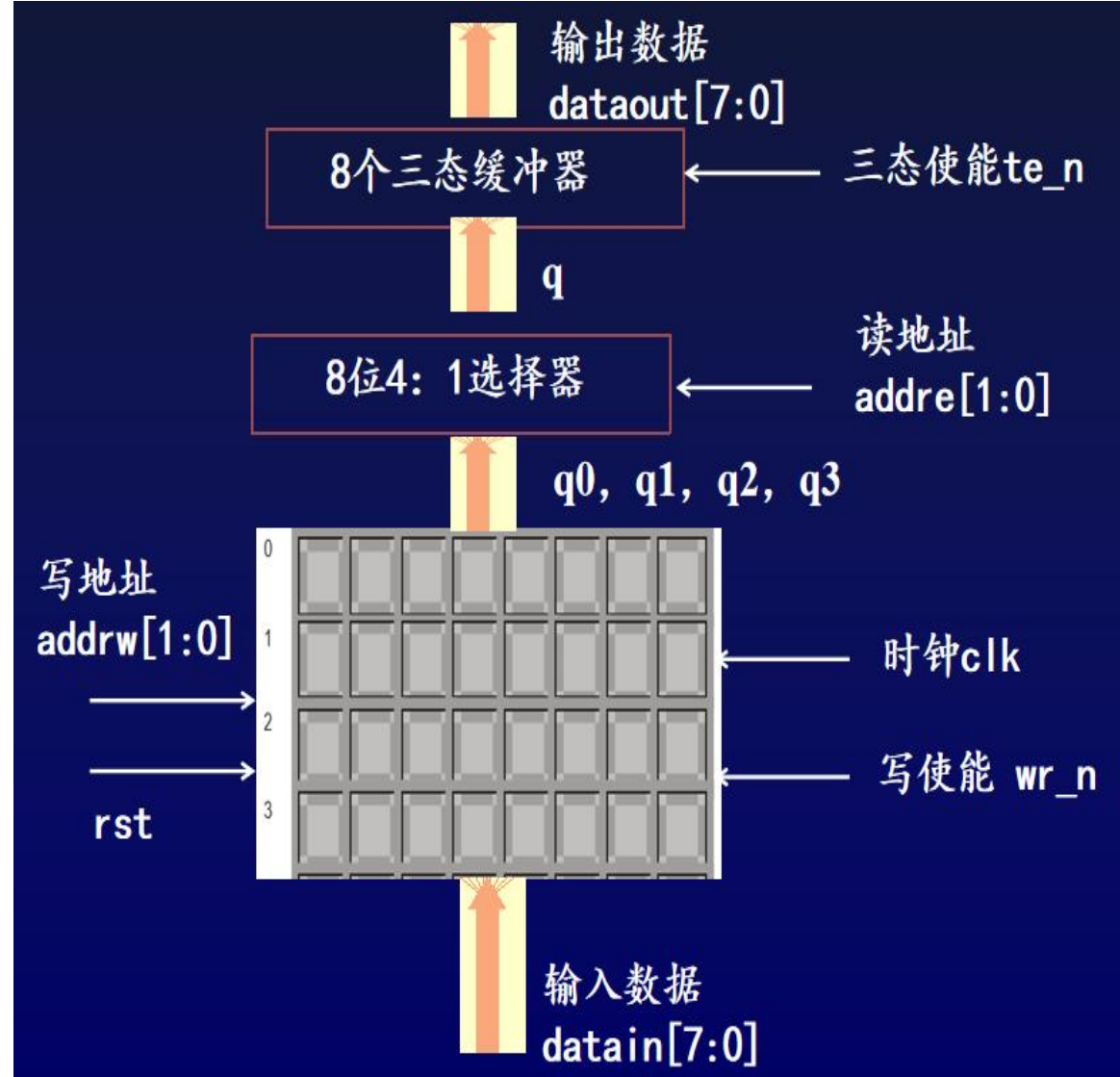
```

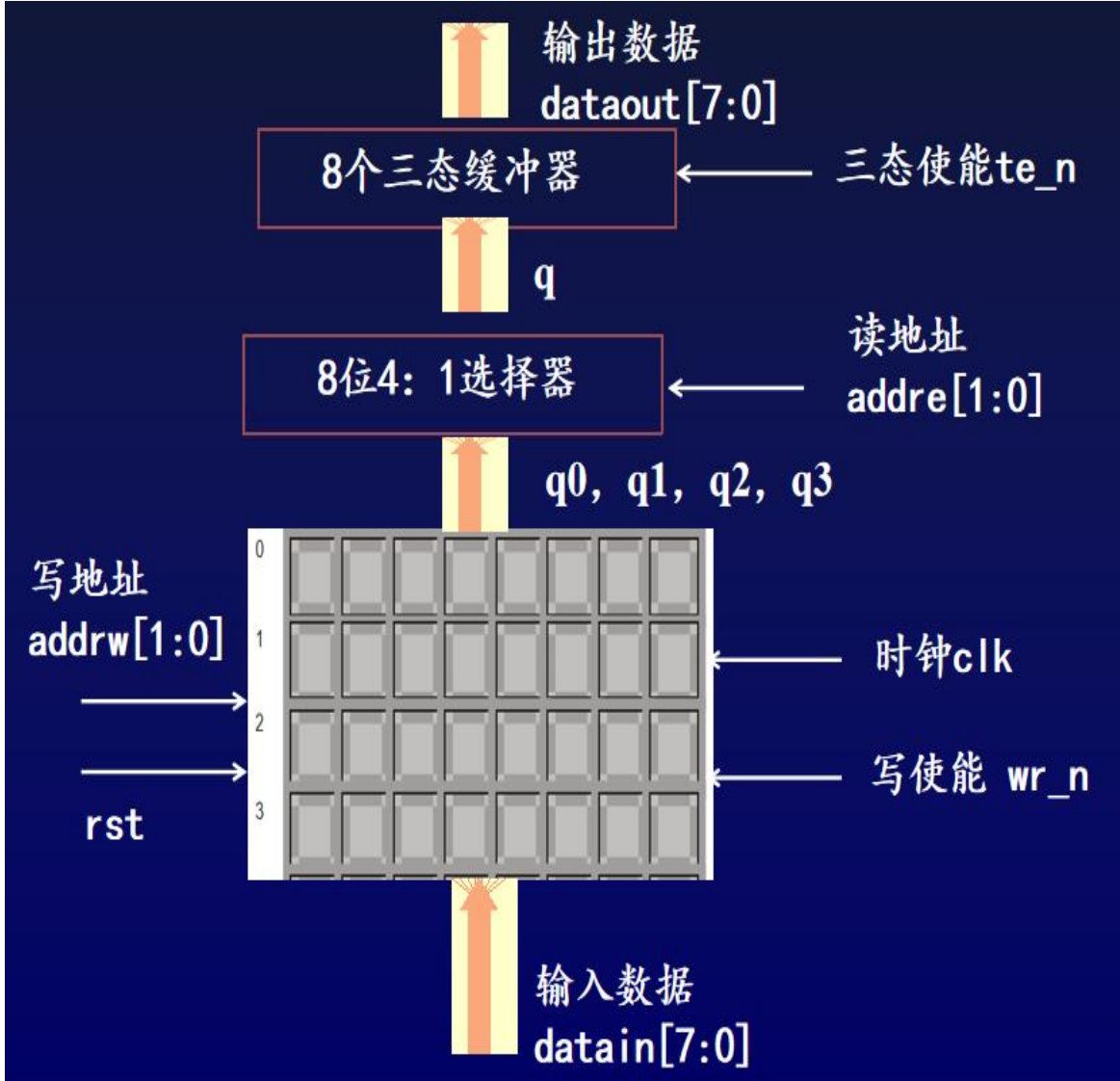
module thi # ( //三态门
    parameter DW=8
) (
    input                te_n ,
    input [DW-1:0]      data ,
    output [DW-1:0]     qout
);

    assign qout = ( !te_n ) ? data : {DW{1'bz}} ;

endmodule

```





```

module regfile # ( //顶层逻辑
    parameter DW = 8 ,
    parameter ADW = 2
) (
    input [DW-1:0]    datain ,
    input [ADW-1:0]   addrw ,
    input [ADW-1:0]   addre ,
    input             clk ,
    input             wr_n ,
    input             te_n ,
    input             rst ,
    output [DW-1:0]    dataout
);

wire [DW-1:0]  q0 , q1 , q2 , q3 , q ;
  
```

```

reg_8 reg_u ( //例化8位寄存器组
    .clk      (clk)      ,
    .wr_n     (wr_n)     ,
    .rst      (rst)      ,
    .addr     (addrw)    ,
    .data     (datain)   ,
    .q0       (q0),
    .q1       (q1),
    .q2       (q2),
    .q3       (q3)
);

```

```

mux4 mux_u ( // 例化4:
1选择器
    (
        .addr  ( addre )    ,
        .d0    ( q0 )       ,
        .d1    ( q1 )       ,
        .d2    ( q2 )       ,
        .d3    ( q3 )       ,
        .f      ( q )
    );

```

```

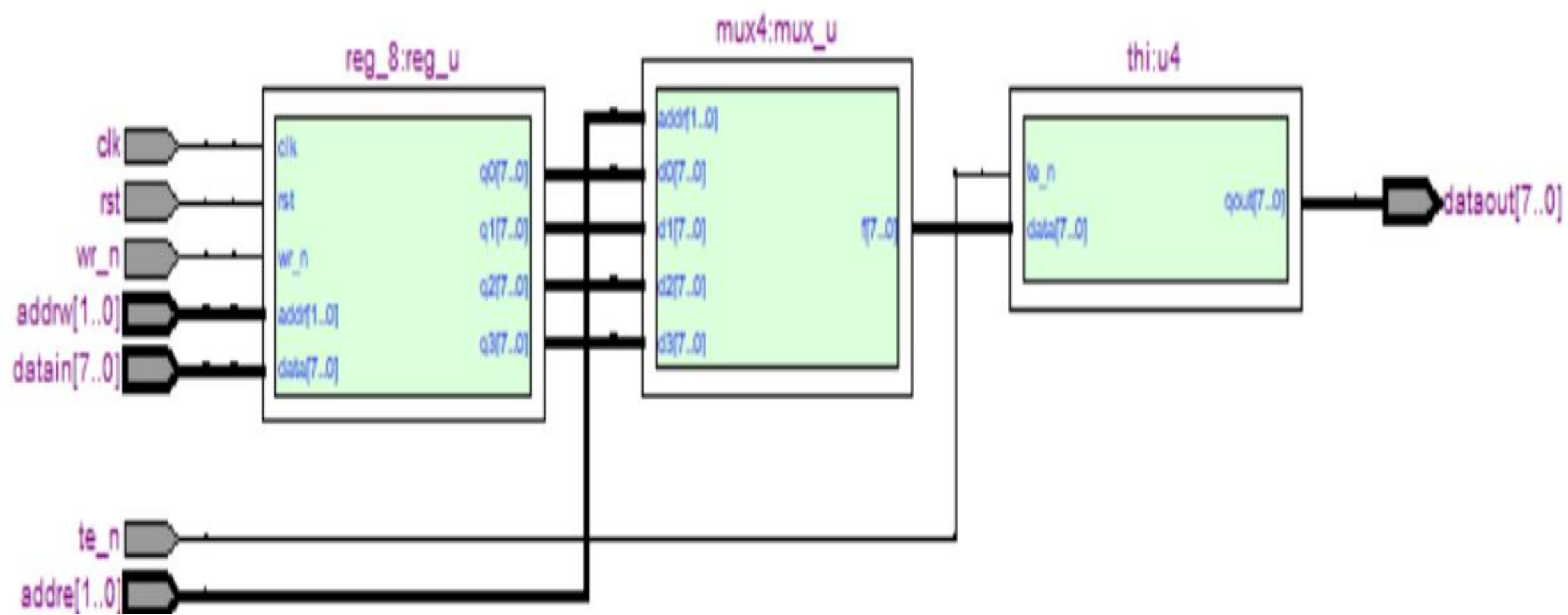
thi u4 ( //例化三态门
    . te_n    ( te_n )      ,
    .data     ( q)           ,
    .qout     (dataout)
);

```

```

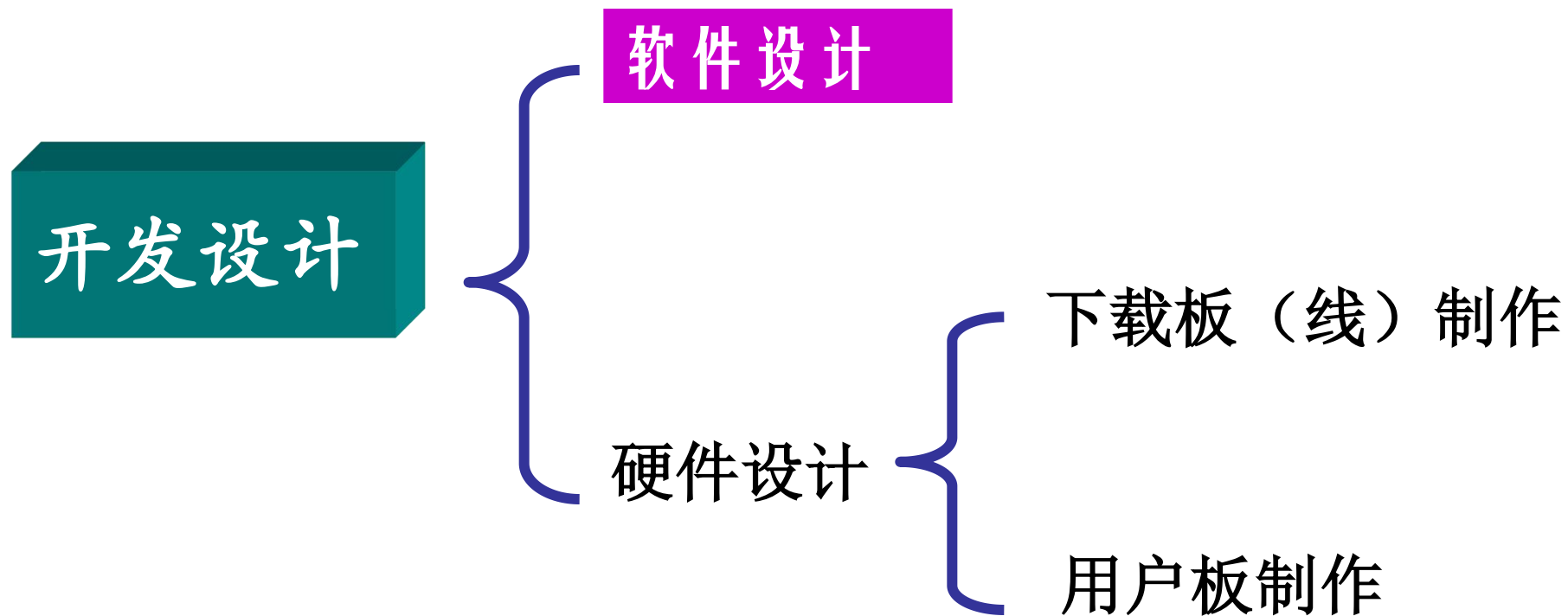
endmodule

```

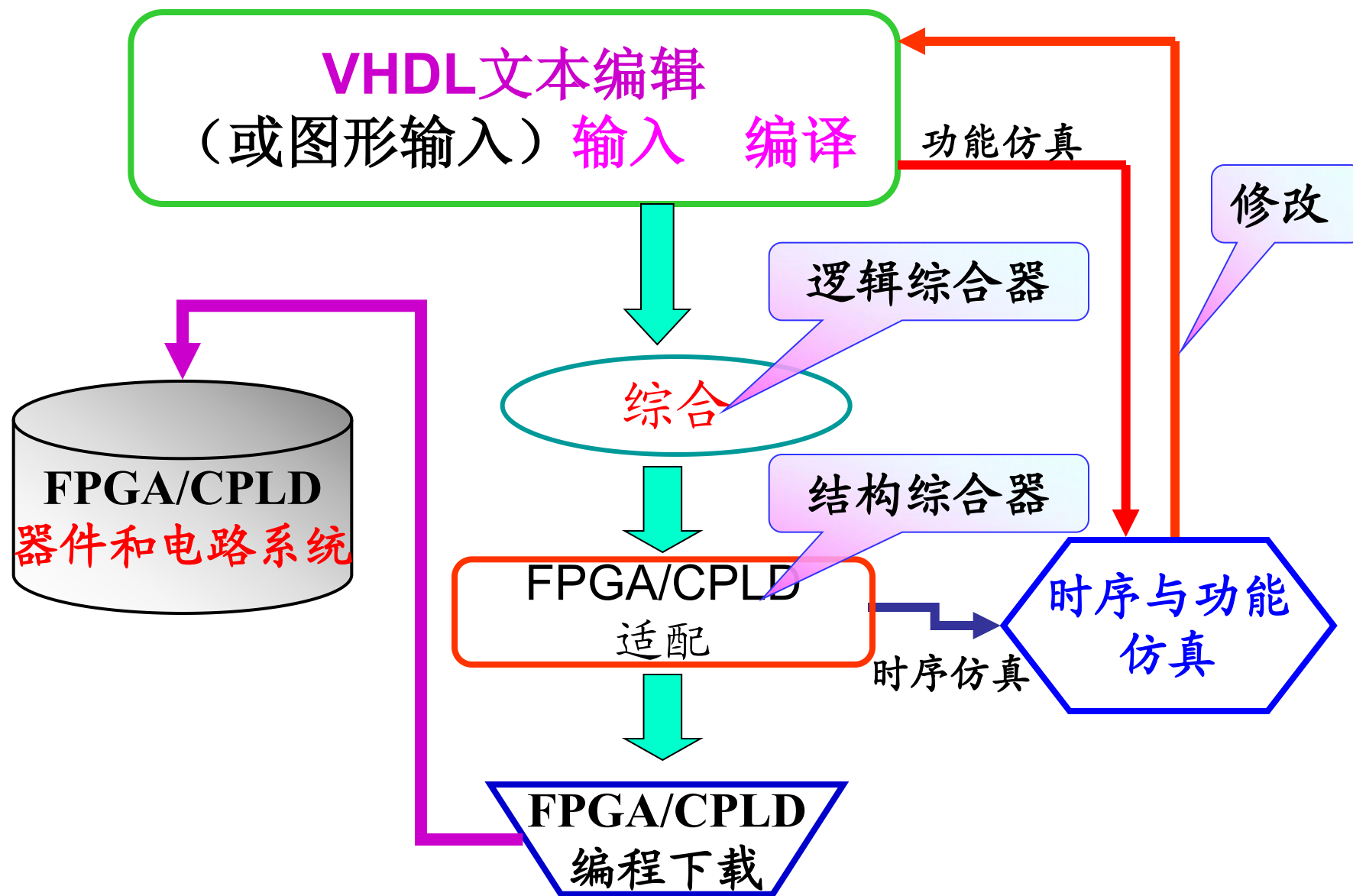


可编程逻辑器件器件的开发流程

开发工具



三、软件设计流程



四.下载芯片（编程）

